10

Solution Techniques for Eigenvalue Problems

10.1 THE GENERALIZED EIGENPROBLEM

In this chapter we are concerned with algorithms for solving the so-called generalized eigenproblem arising from finite element discretizations. Recall that this takes the form: [1]

$$(\boldsymbol{K} - \lambda \boldsymbol{M})\boldsymbol{\psi} = \boldsymbol{0} \tag{10.1.1}$$

where $\boldsymbol{K}$ is symmetric and positive-semidefinite, $\boldsymbol{M}$ is symmetric and positive-definite, and both matrices possess typical band/profile structure. For convenience we recall some basic properties of the solution of (10.1.1). We assume as always that the dimensions of $\boldsymbol{K}$ and $\boldsymbol{M}$ are $n_{eq} \times n_{eq}$. There exist n_{eq} eigenvalues and corresponding eigenvectors [2] that satisfy (10.1.1). That is,

$$\boxed{(\boldsymbol{K} - \lambda_l \boldsymbol{M})\boldsymbol{\psi}_l = 0, \qquad \text{(no sum)}} \tag{10.1.2}$$

where $l = 1, 2, \ldots, n_{eq}$ denotes the mode number. Furthermore,

$$\boxed{0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_{n_{eq}}} \tag{10.1.3}$$

[1] Problems of this type are motivated and formulated in Chapter 7. To simplify subsequent writing we omit the superscript h on λ.

[2] The eigenvectors may be nonunique.

and

$$\boxed{\begin{aligned} \psi_k^T M \psi_l &= \delta_{kl} && (M \text{ orthonormality}) \\ \psi_k^T K \psi_l &= \lambda_l \delta_{kl} \quad \text{(no sum)} && (K \text{ orthogonality}) \end{aligned}} \qquad (10.1.4)\ (10.1.5)$$

The eigenvectors $\{\psi_l\}_{l=1}^{n_{eq}}$ constitute a ***basis*** for $\mathbb{R}^{n_{eq}}$. If, additionally, K is positive definite, as is often the case, then (10.1.3) may be strengthened as follows:

$$0 < \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_{n_{eq}} \qquad (10.1.6)$$

In practice, we are typically interested only in the lower modes. These are the most important from a physical standpoint. For example, the lowest mode is the most important in a structural stability (i.e., "buckling") calculation and the lower frequencies and corresponding mode shapes are usually the most important in considerations of dynamic response. Additionally, the higher modes of finite element discretizations are not accurate renditions of physical behavior but rather spurious artifacts of the discretization process. Consequently, they are of no engineering interest. For these reasons, we are interested in economical computational algorithms for extracting $\{\lambda_l, \psi_l\}$, $1 \leq l \leq n_{\text{modes}}$, where $n_{\text{modes}} \ll n_{eq}$ is the number of desired eigenpairs. In practice n_{eq} may be very large, and thus solution of the eigenvalue problem, even for only a few eigenpairs, may entail an extensive and costly calculation.

Most procedures used for solving the large-scale generalized eigenproblem (10.1.1), involve a ***reduced system*** of the form

$$\boxed{(K^* - \lambda^* M^*)\psi^* = 0} \qquad (10.1.7)$$

where K^* and M^* are *small, full, symmetric matrices*. Algorithms, such as the ***generalized Jacobi method*** [1], are available for directly solving (10.1.7).

Systems such as (10.1.7) may also be solved by first transforming to *standard form*:

$$\boxed{(\bar{K}^* - \lambda^* I)\bar{\psi}^* = 0} \qquad (10.1.8)$$

where

$$\bar{K}^* = \bar{U}^{-T} K^* \bar{U}^{-1} \qquad (10.1.9)$$

$$\bar{\psi}^* = \bar{U} \psi^* \qquad (10.1.10)$$

and $\bar{U}$ is the upper-triangular Cholesky factor of M^*, i.e.,

$$M^* = \bar{U}^T \bar{U} \qquad (10.1.11)$$

(The Cholesky factor is related to the Crout factor by $\bar{U} = D^{1/2}U$, where $M^* = U^T D\, U$, U is upper-triangular with ones on the diagonal and D is a diagonal matrix of positive numbers; see Chapter 11 for further information on the Crout factorization.)

Observe that the eigenvalues of the standard form, (10.1.8), are identical to the generalized form, (10.1.7). However, the eigenvectors need to be transformed as indicated by (10.1.10)

Exercise 1. Derive (10.1.7) from (10.1.8) through (10.1.11).

There are many classical and widely available procedures for solving (10.1.8). For example, the ***Jacobi***, ***Givens***, and ***Householder-QR methods*** may be mentioned in this regard (e.g., see Bathe [1] and Noble [2]). For general matrices, it is currently felt that the most efficient strategy for solving the generalized eigenproblem, (10.1.7), is to first transform to standard form, (10.1.8), and then use the Householder-QR algorithm. However, under certain circumstances, such as when the subspace iteration procedure is employed (see Sec. 10.5), for example, direct use of the generalized Jacobi method proves very effective. It is very difficult to make sweeping statements about efficiency because the type of computer (e.g., sequential, vector, or parallel) strongly influences the performance of algorithms.

Exercise 2. Consider the undamped equation of motion,

$$M\ddot{d} + Kd = F \tag{10.1.12}$$

subject to zero initial displacement and velocity. Expand the solution in terms of the eigenvectors of the associated eigenproblem. Diagonalize the system and exactly solve the individual modal equations. Show that

$$d(t) = \sum_{I=1}^{n_{eq}} \left\{ \frac{1}{\omega_I} \int_0^t F_{(I)}(\tau) \sin \omega_I (t - \tau)\, d\tau\, \psi_I \right\} \tag{10.1.13}$$

The $1/\omega_I$ factor in the expansion illustrates the diminishing influence of the higher modes. This analysis reveals why low mode response is viewed as "most important" and therefore, why, in practical calculations the summation in (10.1.13) is truncated at $n_{\text{modes}} \ll n_{eq}$.

Exercise 3. Obtain an exact solution for the static problem,

$$Kd = F \tag{10.1.14}$$

by way of an eigenvector expansion. Discuss the relative importance of low and high modes for this case.

Exercise 4. Generalize Exercise 2 to account for Rayleigh damping and non-zero initial conditions. Discuss the influence of low and high modes.

10.2 STATIC CONDENSATION

In practice one often encounters eigenvalue problems which can be written in the following *partitioned form*:

$$\left(\begin{bmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{bmatrix} - \lambda \begin{bmatrix} M_{11} & 0 \\ 0 & 0 \end{bmatrix} \right) \begin{Bmatrix} \psi_1 \\ \psi_2 \end{Bmatrix} = 0 \tag{10.2.1}$$

where M_{11} is symmetric and positive-definite. That is, many degrees of freedom are "massless." Problems of this type arise naturally when a relatively light structure is used to support heavy nonstructural masses which can be "lumped" at a few degrees of freedom. In such situations the structural mass is often insignificant and may be neglected, resulting in a system like (10.2.1). As it stands, (10.2.1) is ill-posed in the sense that the zero-diagonal masses give rise to infinite eigenvalues. The mass matrix can be "regularized" by the addition of small positive nonzero diagonal masses, in which case we return to the format of the generalized eigenvalue problem originally considered, or, alternatively, the zero-mass degrees of freedom can be eliminated by *static condensation*. To arrive at the statically condensed form, we expand (10.2.1):

$$K_{11}\psi_1 + K_{12}\psi_2 - \lambda M_{11}\psi_1 = 0 \tag{10.2.2}$$

$$K_{21}\psi_1 + K_{22}\psi_2 = 0 \tag{10.2.3}$$

The next step is to solve (10.2.3) for ψ_2 and then substitute in (10.2.2), which results in

$$(K_{11}^* - \lambda M_{11})\psi_1 = 0 \tag{10.2.4}$$

where

$$K_{11}^* = K_{11} - K_{12}K_{22}^{-1}K_{21} \qquad (\textit{statically condensed stiffness}) \tag{10.2.5}$$

The advantage of transforming to statically condensed form is that the problem size is reduced. However, K_{11}^* tends to be full. Thus, unless the size of the nonzero-mass matrix is rather small, the reduction to statically condensed form may be uneconomical because the profile structure of K is lost.

Note that to calculate the statically condensed stiffness (10.2.5) efficiently, K_{22} is never actually inverted.

The following steps may be used in practice:

$$K_{22} = U^T D U \qquad \text{(Crout factorization)} \tag{10.2.6}$$

$$\boldsymbol{U}\,\boldsymbol{D}^{1/2}\boldsymbol{Z} = \boldsymbol{K}_{21} \qquad \text{(solve for } \boldsymbol{Z}\text{)} \tag{10.2.7}$$

$$\boldsymbol{K}_{12}\boldsymbol{K}_{22}^{-1}\boldsymbol{K}_{21} = \boldsymbol{Z}^T\boldsymbol{Z} \tag{10.2.8}$$

Equation (10.2.7) amounts to solution of an equation system with upper triangular coefficient array and multiple right-hand sides (i.e., the columns of $\boldsymbol{K}_{21}$).

For further computational considerations regarding static condensation, see [3].

10.3 DISCRETE RAYLEIGH-RITZ REDUCTION

In the discrete Rayleigh-Ritz approach, ***static load patterns***, $\boldsymbol{P}$, are selected and corresponding displacement vectors, $\boldsymbol{R}$, are calculated from:

$$\underbrace{\boldsymbol{K}}_{n_{eq}\times n_{eq}}\ \underbrace{\boldsymbol{R}}_{n_{eq}\times n_{lp}} = \underbrace{\boldsymbol{P}}_{n_{eq}\times n_{lp}}, \qquad n_{lp} < n_{eq} \tag{10.3.1}$$

where n_{lp} refers to the number of load patterns. The displacements $\boldsymbol{R}$, referred to as the ***trial vectors***, are used to form the reduced eigenproblem, i.e., (10.1.7) by defining

$$\boldsymbol{K}^* = \boldsymbol{R}^T\boldsymbol{K}\boldsymbol{R} \tag{10.3.2}$$

$$\boldsymbol{M}^* = \boldsymbol{R}^T\boldsymbol{M}\boldsymbol{R} \tag{10.3.3}$$

The load patterns, $\boldsymbol{P}$, are usually subject to the following criteria:

i. The columns of $\boldsymbol{P}$ should be linearly independent.
ii. The columns of $\boldsymbol{P}$ should be selected to arouse the low modes by activating the heaviest masses and most flexible areas of the model.

Remarks

1. If n_{lp} is small, $\boldsymbol{K}^*$ and $\boldsymbol{M}^*$ will be full, but small.

2. Equation (10.3.1) amounts to solution of a multiple right-hand-side system with profile coefficient matrix.

3. The discrete Rayleigh-Ritz procedure is a general formalism for obtaining the reduced system. However, the guidelines for selecting $\boldsymbol{P}$ are somewhat vague and thus a more systematic strategy is required for practical use.

4. The discrete Rayleigh-Ritz reduction is often referred to as a "projection method."

5. The eigenvector approximations are defined by $\boldsymbol{\psi} \cong \boldsymbol{R}\,\boldsymbol{\psi}^*$.

6. By virtue of the fact that the calculation of trial vectors from load patterns requires solution of (10.3.1), $\boldsymbol{K}$ must be nonsingular. This precludes application to cases in which there are zero eigenvalues (e.g., structures which possess rigid body modes). A simple reformulation of the original problem involving a positive *shifting* of the eigenvalues allows us to handle this case: Note that by adding and subtracting $\alpha\boldsymbol{M}\boldsymbol{\psi}$, where α is a positive number, we produce an eigenproblem

$$(\bar{K} - \bar{\lambda}M)\psi = 0 \tag{10.3.}$$

$$\bar{K} = K + \alpha M \tag{10.3.}$$

$$\bar{\lambda} = \lambda + \alpha \tag{10.3.}$$

in which $\bar{K}$ is positive-definite, and the eigenvectors are unchanged. The eigenvalu are related by (10.3.6).

This stratagem may also be employed if some of the λ's are negative. Suppo an estimate of the smallest eigenvalue, λ_1, is available. Then select $\alpha > 0$ such th $-\alpha < \lambda_1 \leq 0$. The $\bar{\lambda}$'s are then all positive and solution may proceed as usual.

7. Shifting is a particular example of a general class of ***spectral tran formations***. Let

$$S = [\psi_1, \psi_2, \ldots, \psi_{n_{eq}}] \tag{10.3.}$$

and

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_{n_{eq}}) \tag{10.3.}$$

Then it is easily verified that

$$K = S \Lambda S^T \tag{10.3.}$$

and

$$M = SS^T \tag{10.3.1}$$

Let $f = f(\lambda)$ represent a scalar-valued function of λ. Define a matrix-valued functi $f = f(K)$ by way of

$$f(K) = S \,\text{diag}(f(\lambda_1), f(\lambda_2), \ldots, f(\lambda_{n_{eq}}))\, S^T \tag{10.3.1}$$

Then it is readily established that solutions of the eigenproblem

$$(f(K) - \bar{\lambda}M)\phi = 0 \tag{10.3.1}$$

are related to solutions of

$$(K - \lambda M)\psi = 0 \tag{10.3.1.}$$

by

$$\bar{\lambda} = f(\lambda) \tag{10.3.1}$$

$$\phi = \psi \tag{10.3.1.}$$

Exercise 1. Consider the partitioned eigenproblem defined by (10.2.1). Show that the sta ically condensed eigenproblem, (10.2.4) and (10.2.5), can be obtained from the discre Rayleigh-Ritz approach by selecting the trial vectors to be

$$R = \begin{bmatrix} I \\ \hline -K_{22}^{-1} K_{21} \end{bmatrix} \tag{10.3.1}$$

10.4 IRONS-GUYAN REDUCTION

The Irons-Guyan reduction [4–6] fits into the discrete Rayleigh-Ritz framework described in the previous section. To understand the method it is helpful to rewrite the generalized eigenproblem in partitioned form:

$$\left(\begin{bmatrix} \boldsymbol{K}_{11} & \boldsymbol{K}_{12} \\ \boldsymbol{K}_{21} & \boldsymbol{K}_{22} \end{bmatrix} - \lambda \begin{bmatrix} \boldsymbol{M}_{11} & \boldsymbol{M}_{12} \\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22} \end{bmatrix} \right) \begin{Bmatrix} \boldsymbol{\psi}_1 \\ \boldsymbol{\psi}_2 \end{Bmatrix} = \mathbf{0} \tag{10.4.1}$$

The degrees of freedom in $\boldsymbol{\psi}_1$ are to be retained in the reduced eigenproblem, whereas those in $\boldsymbol{\psi}_2$ are to be eliminated. For these purposes $\boldsymbol{R}$ is defined by (10.3.16), which leads to the definitions of the reduced arrays (verify!):

$$\boldsymbol{K}^* = \boldsymbol{R}^T \boldsymbol{K} \boldsymbol{R} = \boldsymbol{K}_{11} - \boldsymbol{K}_{12}\boldsymbol{K}_{22}^{-1}\boldsymbol{K}_{21} \tag{10.4.2}$$

$$\boldsymbol{M}^* = \boldsymbol{R}^T \boldsymbol{M} \boldsymbol{R} = \boldsymbol{M}_{11} - \boldsymbol{M}_{12}\boldsymbol{K}_{22}^{-1}\boldsymbol{K}_{21} - \boldsymbol{K}_{12}\boldsymbol{K}_{22}^{-1}(\boldsymbol{M}_{21} - \boldsymbol{M}_{22}\boldsymbol{K}_{22}^{-1}\boldsymbol{K}_{21}) \tag{10.4.3}$$

Note that (10.4.2) is the statically condensed stiffness and, if $\boldsymbol{M}_{22} = \mathbf{0}$ and $\boldsymbol{M}_{12} = \boldsymbol{M}_{21}^T = \mathbf{0}$, then $\boldsymbol{M}^* = \boldsymbol{M}_{11}$ as in the statically condensed problem of Sec. 10.2.

The Irons-Guyan reduction may be thought of as invoking a "static relation," namely

$$\boldsymbol{\psi}_2 = -\boldsymbol{K}_{22}^{-1}\boldsymbol{K}_{21}\boldsymbol{\psi}_1 \tag{10.4.4}$$

to eliminate the unwanted variables. Sometimes $\boldsymbol{\psi}_1$ and $\boldsymbol{\psi}_2$ are referred to as the "master" and "slave" degrees of freedom, respectively. In practice, the following guidelines are suggested:

i. Determine the number of eigenvalues and eigenvectors required, say n_{modes}.
ii. Retain a multiple of n_{modes} degrees of freedom (e.g., $5 \times n_{\text{modes}}$ degrees of freedom).
iii. The degrees of freedom to be retained are identified by calculating the ratios of diagonal elements in $\boldsymbol{M}$ and $\boldsymbol{K}$, namely, K_{pp}/M_{pp} $1 \le p \le n_{eq}$. The degrees of freedom with the smallest ratios are retained (i.e., they are the degrees of freedom in $\boldsymbol{\psi}_1$; [7]).

Remark
The calculation of the reduced mass, (10.4.3), is costly!

10.5 SUBSPACE ITERATION

A disadvantage of reduction techniques such as the Irons-Guyan procedure is that there is no guarantee that the eigenvalues and eigenvectors of the reduced problem,

namely, λ_l^* and $R\psi_l^*$, will be good approximations of those of the original problem, λ_l and ψ_l, respectively. Consequently, methods have been developed in which a reduced problem is used along with an iterative strategy to obtain exactly the lower modes of the generalized eigenproblem. This is the underlying idea of the ***subspace iteration,*** or ***block power, method,*** which is widely used for large-scale finite element calculations. Roughly speaking, the procedure is as follows: Load patterns are selected and trial vectors are calculated. The trial vectors are used to form a reduced problem, which is solved. New load patterns are calculated from the "inertial" forces engendered by the eigenvectors of the reduced problem, namely,

$$\underbrace{P}_{n_{eq} \times n_{lp}} = MR[\psi_1^*, \psi_2^*, \ldots, \psi_{n_{lp}}^*] \tag{10.5.1}$$

With these load patterns the process is repeated until convergence is achieved. The calculations are summarized in the flowchart contained in Table 10.5.1. Extensive discussion and further details are presented in [8].

TABLE 10.5.1 Subspace Iteration Procedure

I. Initialization
   1. Form K and M.
   2. Factorize $K = U^T DU$.
   3. Specify initial load patterns, P.

II. Iteration
   1. Solve for trial vectors:
   $$(U^T D\, U)R = P$$
   2. Compute reduced matrices:
   $$K^* = R^T KR = R^T P$$
   $$M^* = R^T MR$$
   3. Solve the reduced eigenproblem:
   $$(K^* - \lambda_k^* M^*)\psi_k^* = 0, \qquad k = 1, 2, \ldots, n_{lp}$$
   4. Calculate approximations to the eigenvectors of the original system:
   $$R[\psi_1^*, \psi_2^*, \ldots, \psi_{n_{lp}}^*]$$
   5. Perform convergence checks. If the desired eigenvalues have converged, stop. Otherwise continue.
   6. Calculate improved load patterns:
   $$P = MR[\psi_1^*, \psi_2^*, \ldots, \psi_{n_{lp}}^*]$$
   7. Go to Step II-1.

Remarks

1. It is recommended from practical experience [8] that:

i. The number of load patterns employed should be calculated from

$n_{lp} = \min \{2\, n_{\text{modes}},\ n_{\text{modes}} + 8\}$, where n_{modes} is the number of eigenpairs desired.

ii. In the first load pattern (i.e., first column of $\boldsymbol{P}$) a 1 should be placed in the entry corresponding to the minimum value of K_{pp}/M_{pp}, and 0 in the remaining entries. Likewise, a 1 should be placed in the entry of the second column corresponding to the next smallest value of K_{pp}/M_{pp}, and so forth.

2. Wilson [9] recommends that one load pattern be generated from random numbers in the interval [0, 1].

3. The convergence condition may be specified in terms of consecutive iterates:

$$\frac{(\lambda_k^*)_{I+1} - (\lambda_k^*)_I}{(\lambda_k^*)_I} \leq \epsilon, \qquad k = 1, 2, \ldots, n_{\text{modes}}$$

where I is the iteration number and ϵ is a preassigned error tolerance.

4. Solution of the generalized eigenproblem in the subspace iteration algorithm (Step II-3 in Table 10.5.1) is efficiently carried out by way of the ***generalized Jacobi method*** [8]. The reason for this is that after a number of iteration steps, the generalized eigenproblem possesses diagonally dominant matrices for which the generalized Jacobi method proves very effective.

10.5.1 Spectrum Slicing

The convergence of a procedure such as subspace iteration does not necessarily guarantee that the first n_{modes} eigenpairs of the original system have been found. An eigenvalue can be missed if the original trial vectors are orthogonal to the corresponding eigenvector. To ascertain whether or not this has occurred, a ***spectrum slicing*** may be performed.[3] The steps involved are as follows: Perform a Crout factorization of the matrix $\boldsymbol{K} - \alpha \boldsymbol{M}$, where $\alpha = (1 + \delta)\lambda_{n_{\text{modes}}}$ and δ is a small positive number such that $(1 + \delta)\lambda_{n_{\text{modes}}} < \lambda_{n_{\text{modes}+1}}$. Let $\boldsymbol{D}_\alpha$ denote the diagonal matrix of pivots obtained. It follows from ***Sylvester's inertia theorem*** that the number of eigenvalues smaller than α will equal the number of negative entries in $\boldsymbol{D}_\alpha$. If this number, say n_α, is greater than n_{modes}, then $n_\alpha - n_{\text{modes}}$ eigenvalues smaller than α were missed in the calculation. If this is the case, a revised set of load patterns must be employed and the eigensolution repeated. If $n_\alpha = n_{\text{modes}}$, then a degree of confidence in the solution is attained.

Remark

Spectrum slicing can be used to determine the number of eigenvalues in an interval $]\alpha, \beta[$, where $\beta > \alpha$. Factorize $\boldsymbol{K} - \alpha \boldsymbol{M}$ and $\boldsymbol{K} - \beta \boldsymbol{M}$. If $\boldsymbol{D}_\alpha$ and $\boldsymbol{D}_\beta$ are the corresponding pivots and n_α and n_β are the number of negative entries of $\boldsymbol{D}_\alpha$ and $\boldsymbol{D}_\beta$, respectively, then $n_\beta - n_\alpha$ is the number of eigenvalues in $]\alpha, \beta[$.

[3] Spectrum slicing is sometimes referred to as a ***Sturm sequence check*** [8].

10.5.2 Inverse Iteration

When the number of load patterns used in the subspace iteration procedure is one, the process is called ***inverse iteration***. In this case convergence to the lowest eigenvalue occurs as long as the initial trial vector is not orthogonal to the corresponding eigenvector.

By reversing the roles of K and M, convergence to the largest eigenvalue, $\lambda_{n_{eq}}$, can be achieved. This may be seen from:

$$\mathbf{0} = (K - \lambda M)\psi = (M - \lambda^{-1}K)\psi$$

Note that when working with the latter form, subspace iteration requires M to be nonsingular.

Exercise 1. Consider the following eigenvalue problem:

$$(K - \lambda M)\psi = \mathbf{0}, \qquad \lambda = \omega^2$$

where

$$M = \begin{bmatrix} m_1 & 0 \\ 0 & m_2 \end{bmatrix}$$

$$K = \begin{bmatrix} (k_1 + k_2) & -k_2 \\ -k_2 & k_2 \end{bmatrix}$$

$$\psi = \begin{Bmatrix} \psi_1 \\ \psi_2 \end{Bmatrix}$$

Assume $k_1 = k_2 = 1$, $m_1 = 3$, $m_2 = 2$.

a. Calculate the frequencies (i.e., ω_1, ω_2) and mode shapes (i.e., ψ_1, ψ_2).

b. Assuming that the fundamental mode load pattern is given approximately by

$$P = \begin{Bmatrix} \dfrac{m_1}{(k_1 + k_2)} \\ \dfrac{m_2}{k_2} \end{Bmatrix}$$

use the discrete Rayleigh-Ritz reduction procedure to obtain an estimate of the fundamental frequency and mode shape.

c. Use the Irons-Guyan procedure to reduce the problem to one degree of freedom. Pick the degree of freedom to be retained according to the criterion presented in Sec. 10.4. Determine the approximate fundamental frequency and mode shape.

d. Use the subspace iteration procedure to calculate the fundamental frequency and mode shape. Initialize the computations with the load pattern

$$P = \begin{Bmatrix} 0 \\ 1 \end{Bmatrix}$$

Employ two iterations.

Solution

a. $$\boldsymbol{M} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \qquad \boldsymbol{K} = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$$

$$0 = \det \begin{bmatrix} 2 - 3\lambda & -1 \\ -1 & 1 - 2\lambda \end{bmatrix} = 6\lambda^2 - 7\lambda + 1$$

$$\lambda_{1,2} = \tfrac{1}{6},\ 1$$

$$\omega_{1,2} = \frac{1}{\sqrt{6}},\ 1 = 0.40825,\ 1$$

$$\psi_1 = \begin{Bmatrix} \frac{2}{3} \\ 1 \end{Bmatrix} \qquad \psi_2 = \begin{Bmatrix} 1 \\ -1 \end{Bmatrix}$$

b. $$\boldsymbol{P} = \begin{Bmatrix} \frac{3}{2} \\ 2 \end{Bmatrix}$$

$$\boldsymbol{KR} = \boldsymbol{P}$$

$$\boldsymbol{R} = \begin{Bmatrix} \frac{7}{2} \\ \frac{11}{2} \end{Bmatrix} \qquad \text{(the } \frac{1}{2} \text{ factors may be neglected)}$$

$$\boldsymbol{K}^* = \boldsymbol{R}^T\boldsymbol{K}\boldsymbol{R} = \langle 7 \ \ 11 \rangle \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix} \begin{Bmatrix} 7 \\ 11 \end{Bmatrix} = 65$$

$$\boldsymbol{M}^* = \boldsymbol{R}^T\boldsymbol{M}\boldsymbol{R} = \langle 7 \ \ 11 \rangle \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \begin{Bmatrix} 7 \\ 11 \end{Bmatrix} = 389$$

$$(\boldsymbol{K}^* - \lambda^*\boldsymbol{M}^*)\psi^* = \boldsymbol{0} \Rightarrow \lambda^* = \frac{65}{389}, \qquad \psi^* = 1$$

$$\psi_1 \cong \boldsymbol{R}\psi_1^* = \frac{1}{11}\begin{Bmatrix} 7 \\ 11 \end{Bmatrix} = \begin{Bmatrix} 0.64 \\ 1 \end{Bmatrix}; \qquad \omega^* = \left(\frac{65}{389}\right)^{1/2} = 0.4087$$

c. $$\left.\begin{aligned} \frac{K_{11}}{M_{11}} &= \frac{2}{3} \\ \frac{K_{22}}{M_{22}} &= \frac{1}{2} \end{aligned}\right\} \Rightarrow \text{retain degree of freedom number 2}$$

Reorder equations into the standard partitioned form.

$$\boldsymbol{K} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix} \qquad \boldsymbol{M} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

$$\boldsymbol{R} = \begin{Bmatrix} 1 \\ -\boldsymbol{K}_{22}^{-1}\boldsymbol{K}_{21} \end{Bmatrix} = \begin{Bmatrix} 1 \\ \frac{1}{2} \end{Bmatrix}$$

$$\boldsymbol{K}^* = \boldsymbol{R}^T\boldsymbol{K}\boldsymbol{R} = \frac{1}{2}$$

$$\boldsymbol{M}^* = \boldsymbol{R}^T\boldsymbol{M}\boldsymbol{R} = \frac{11}{4}$$

$$\lambda^* = \frac{2}{11}, \qquad \boxed{\omega^* = \left(\frac{2}{11}\right)^{1/2} = 0.4264}, \qquad \psi_1^* = 1$$

$$\psi_1 \cong \boldsymbol{R}\,\psi_1^* = \begin{Bmatrix} 1 \\ \frac{1}{2} \end{Bmatrix}, \qquad \text{(reorder):} \qquad \boxed{\psi_1 = \begin{Bmatrix} \frac{1}{2} \\ 1 \end{Bmatrix}}$$

d. $\boldsymbol{P} = \begin{Bmatrix} 0 \\ 1 \end{Bmatrix}$; $\boldsymbol{K}\,\boldsymbol{R} = \boldsymbol{P}$; $\boldsymbol{R} = \begin{Bmatrix} 1 \\ 2 \end{Bmatrix}$

$\boldsymbol{K}^* = 2$; $\boldsymbol{M}^* = 11$; $\lambda^* = \frac{2}{11}$ (same as (c)) } iteration number 1

$\omega^* = 0.4264$ $\psi_1 \cong \begin{Bmatrix} \frac{1}{2} \\ 1 \end{Bmatrix}$ }

$\boldsymbol{P} = \boldsymbol{M}\,\psi_1 = \begin{Bmatrix} \frac{3}{2} \\ 2 \end{Bmatrix}$ (same as (b)) }

Therefore } iteration number 2

$\omega^* = 0.4087$ $\psi_1 \cong \begin{Bmatrix} 0.64 \\ 1 \end{Bmatrix}$ }

THE LANCZOS ALGORITHM FOR SOLUTION OF LARGE GENERALIZED EIGENPROBLEMS

BAHRAM NOUR-OMID

10.6.1 Introduction

The Lanczos algorithm was first proposed in 1950. Lanczos intended his algorithm to be used for computing a few of the extreme eigenvalues and corresponding eigenvectors of a symmetric matrix. However, the algorithm was taken up as a method for reducing a symmetric matrix to tridiagonal form. Lanczos himself observed that round-off error has a significant effect on the algorithm's performance and expensive modifications seemed to be necessary to overcome this defect. By 1955 the Householder method replaced the Lanczos algorithm as a more efficient method for tridiagonalizing a matrix.

Most engineering applications require only a few eigenvalues at one end of the spectrum, but for small problems the Householder-QR algorithm can find all the eigenvalues almost as fast as a few. For large problems however it is a waste to tridiagonalize the whole matrix. The Lanczos iteration has the property that well-isolated eigenvalues are approximated accurately after a comparatively small number of steps. For example, the largest eigenvalues will often be captured after 20 steps, almost independently of the order of the problem, n. Moreover, the number of steps taken by the Lanczos method to compute these eigenvalues will always be less than that for the power method. This property makes the Lanczos algorithm very efficient. However, the eigenvalues most frequently wanted are the smaller ones. The Lanczos algorithm is so powerful that it can compute the smaller eigenvalues of a matrix without any factorization. However, they will not be well approximated until nearly n steps have been taken. Consequently it is necessary, in these cases, to apply the iteration to an inverted form of the matrix. In this respect the Lanczos algorithm is just like the subspace iteration method. See [15] for a comparison of Lanczos and subspace iteration algorithms.

In this section we are concerned with the application of the Lanczos algorithm to the generalized symmetric eigenproblem

$$(\boldsymbol{K} - \lambda \boldsymbol{M})z = \mathbf{0} \tag{10.6.1}$$

where $\boldsymbol{K}$ and $\boldsymbol{M}$ are $n \times n$ real symmetric matrices. We assume for the moment that the eigenvalues of (10.6.1) are real. Later we state the conditions on $\boldsymbol{K}$ and $\boldsymbol{M}$ that must hold to ensure real eigenvalues. We will derive the generalized Lanczos algorithm for the solution of the eigenproblem (10.6.1). The relation between the Lanczos method and vector iteration methods will be established. We then look at the effect of round-off on the algorithm and the resulting loss of orthogonality. We consider two possible modifications to the algorithm that can maintain a desired level of orthogonality among the Lanczos vectors. Finally, we give a detailed description of the algorithm we prefer together with a listing of the computer subprograms.

10.6.2 Spectral Transformation

We turn now to an important misconception concerning the Lanczos algorithm. It is usually thought of as a way of computing eigenpairs, (λ, z), of the standard eigenproblem

$$(A - \lambda I)z = 0 \tag{10.6.2}$$

The Lanczos method is so powerful that one can work directly with A to evaluate eigenvalues at both ends of the spectrum of (10.6.2) without solving any system of equations. Only products of A with a sequence of vectors need be computed. This virtue blinded certain users of the Lanczos method to the great advantages to be gained by a shift-and-invert procedure. The matrix $(A - \sigma I)^{-1}$ has the same eigenvectors as A. Using $(A - \sigma I)^{-1}$ instead of A will cause eigenvectors corresponding to eigenvalues close to σ to converge much more quickly. Of course there are cases when factoring of A is not possible (e.g., when the matrix is only known implicitly) or not desirable (e.g., when A has a given sparsity structure that can be destroyed when factored).

However, we are more interested in the generalized eigenproblem (10.6.1). For this problem some form of inversion or factoring of a matrix, either explicitly or implicitly, is required. It is interesting to note that when the structure of M is the same as that of K, as in the case of a consistent mass matrix, then the cost of one step of the power iteration method is exactly the same as that for the inverse iteration method. Each requires one triangular factorization.

There are two different procedures for transforming the generalized eigenproblem to the standard form:

i. Factor M into $M = CC^T$. This is the Cholesky factorization of M. Then

$$(K - \lambda M) = C(C^{-1}KC^{-T} - \lambda I)C^T \tag{10.6.3}$$

and the eigenproblem of (10.6.1) reduces to the standard form $(A_1 - \lambda I)\hat{z} = 0$ where $A_1 = C^{-1}KC^{-T}$ and $\hat{z} = C^T z$. The eigenvalues of the standard problem are the same as those of the generalized problem. Note that if M is positive-semidefinite, then its Cholesky factors are singular and this transformation can not be performed.

ii. Using a similar procedure, (10.6.1) can be transformed into

$$(A_2 - \mu I)\hat{z} = 0 \tag{10.6.4}$$

where $A_2 = C^T K^{-1} C$ and $\mu = 1/\lambda$. Note that in this case both the eigenvalues and the eigenvectors of A_2 are different from those of (10.6.1).

A more general form of the second transformation is to first perform a shift of the origin, $(K_\sigma - (\lambda - \sigma)M)z = 0$, where $K_\sigma = K - \sigma M$, and then perform (ii). This results in a standard eigenproblem with $A_\sigma = C^T K_\sigma^{-1} C$. The spectrum of A_σ is related to the original spectrum through

$$\boxed{\nu = \frac{1}{\lambda - \sigma}} \tag{10.6.5}$$

where ν is the eigenvalue of A_σ[11].

The second procedure requires two triangular factorizations, one for M and one for K. It is possible to avoid the factorization of M by working with

$$(K_\sigma^{-1}M - \nu I)z = 0 \tag{10.6.6}$$

Although the matrix of this transformation is not symmetric, it is self-adjoint with respect to the ***inertial inner product*** defined by

$$(u,v)_M = v^T M u \tag{10.6.7}$$

which is shown in the following steps:

$$\begin{aligned}(K_\sigma^{-1}Mu,\ v)_M &= v^T M\, K_\sigma^{-1}Mu \\ &= (u,\ K_\sigma^{-1}Mv)_M\end{aligned}$$

This unsymmetric form is particularly advantageous since it has the same eigenvectors as the original problem [16]. The algorithm that is derived later employs the transformation of equation (10.6.6).

10.6.3 Conditions for Real Eigenvalues

Contrary to common belief, the fact that K and M are symmetric is not a sufficient condition to ensure real eigenvalues for (10.6.1). This can be illustrated using the following simple example.

Example 1

Let $K = \begin{bmatrix}1 & 1\\ 1 & 0\end{bmatrix}$ and $M = \begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}$. Then the eigenvalues of (10.6.1) with these matrices are $\frac{1}{2}(1 \pm i\sqrt{3})$, where $i^2 = -1$.

The eigenvalues of (10.6.1) are all real if some linear combination of K and M is positive-definite; that is $\rho K + \tau M$ is positive-definite for some choice of real ρ and τ. This is only a sufficient condition that guarantees real eigenvalues, but there is no easy way of computing ρ and τ.

Exercise 1. Prove that $\rho K + \tau M$ being positive definite for some choice of ρ and τ is only a sufficient condition for (10.6.1) to have real eigenvalues.

Solution Multiply (10.6.1) by ρ and shift the spectrum by τ:

$$\begin{aligned}0 &= (\rho K - \rho\lambda M)z \\ &= (\rho K + \tau M - (\rho\lambda + \tau)M)z\end{aligned}$$

Because $\rho K + \tau M$ is positive definite, it possesses a Cholesky decomposition; i.e., $\rho K + \tau M = CC^T$. Thus, the above equation can be reduced to standard form:

$$(A - \mu I)\hat{z} = 0$$

where $A = C^{-1}MC^{-T}$, $\hat{z} = C^T z$, and $\mu = 1/(\rho\lambda + \tau)$. By the symmetry of A, the μ's

are real. Consequently, so are the λ's. This establishes "sufficiency". Now consider the following case. Let

$$K = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 3 \end{bmatrix}, \quad \text{and} \quad M = \begin{bmatrix} -3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

Clearly, the eigenvalues of this problem are real. However, there is no linear combination of K and M that is positive definite. This establishes that the above is not a necessary condition.

Fortunately, in many problems encountered in structural mechanics, either K or M or both are positive-definite.

10.6.4 The Rayleigh-Ritz Approximation

Consider a given set of vectors, $X_m = [x_1, x_2, \cdots, x_m]$, with $m \ll n$. We refer to these as ***trial vectors***. We proceed to obtain an approximation to some of the eigenvectors of (10.6.1) by taking a linear combination of the trial vectors. Let $y = X_m s = \sum_{i=1}^{m} x_i s_i$ be the desired approximation. The ith component of s is the coefficient of x_i in this representation of y. We denote the corresponding approximation to the eigenvalues of (10.6.1) by θ. The ***residual***, r, associated with the approximating pair $\{\theta, y\}$ is given by

$$\boxed{r = Ky - \theta My} \tag{10.6.8}$$

The Rayleigh-Ritz method requires the residual vector be orthogonal to each of the trial vectors; i.e.,

$$X_m^T r = X_m^T K y - \theta X_m^T M y = 0$$

Substituting the representation of y in terms of the trial vectors, in the above orthogonality condition for r, we obtain the reduced eigenproblem

$$(K_m - \theta M_m)s = 0$$

where $K_m = X_m^T K X_m$ and $M_m = X_m^T M X_m$. The eigenvector of the reduced eigenproblem, $s^{(k)}$, will determine the approximation, $y^{(k)}$, to the eigenvector of (10.6.1). We refer to $y^{(k)}$ as the ***Ritz vector*** and to its corresponding $\theta^{(k)}$ as the ***Ritz value***, for each $k = 1, \cdots, m$.[4]

[4] Some authors refer to the trial vectors as Ritz vectors. In keeping with more prevalent usage, we prefer to reserve the latter terminology for the y's.

10.6.5 Derivation of The Lanczos Algorithm

The Lanczos method can be thought of as a means of constructing an orthogonal set of vectors, known as Lanczos vectors, for use in the Rayleigh-Ritz procedure. The algorithm is closely related to the inverse iteration and power methods for calculating a single eigenpair. Given a pair of matrices $K_\sigma = K - \sigma M$ and M, and a starting vector r, these basic methods generate a sequence of vectors, $\{r, K_\sigma^{-1}Mr, (K_\sigma^{-1}M)^2 r, \ldots, (K_\sigma^{-1}M)^j r\}$, during j iterations. These vectors are referred to as the ***Krylov sequence***; the sequence converges (as $j \to \infty$) to the eigenvector corresponding to the eigenvalue, λ, of (10.6.1) closest to the shift σ.

The basic difference between the Lanczos method and the other two is that the information contained in each successive vector of the Krylov sequence is used to obtain the best approximation to the wanted eigenvectors instead of using only the last vector in the sequence. To be more specific, the Lanczos algorithm is equivalent to obtaining the Rayleigh-Ritz approximation with the vectors in the Krylov sequence as the trial vectors. This involves supplementing the Krylov sequence with the Gram-Schmidt orthogonalization process at each step. The result is a set of M-orthonormal vectors (the ***Lanczos vectors***) that is used in the Rayleigh-Ritz procedure to reduce the dimension of the eigenproblem. We show below that orthogonalization is required only with respect to two preceding vectors; a fact recognized by Lanczos. The Rayleigh-Ritz procedure with the M-orthonormal basis of the Krylov subspace leads to a standard eigenproblem with a tridiagonal matrix.

To derive the Lanczos algorithm it will be assumed for the moment that the first j Lanczos vectors, $\{q_1, q_2, \cdots, q_j\}$ have been found, and the construction of the $j + 1$ vector will be described. The resulting vectors all satisfy the condition $q_i^T M q_j = \delta_{ij}$, where δ_{ij} is the Kronecker delta; that is, the vectors are orthonormal with respect to the mass matrix. To calculate q_{j+1}, we must orthogonalize $v_j = (K_\sigma^{-1}M)^j r$ against the j Lanczos vectors computed so far. From the definition of v_j we obtain $v_j = K_\sigma^{-1} M v_{j-1}$. Now, v_{j-1} is the vector that is M-orthonormalized against the first $j - 1$ Lanczos vectors to obtain q_j. Therefore

$$v_{j-1} = \sum_{i=1}^{j} v_i q_i$$

where v_i is the component of v_{j-1} along q_i. This result is used to eliminate v_{j-1} in the above recursive relation for v_j. We then get

$$\begin{aligned} v_j &= \sum_{i=1}^{j} v_i K_\sigma^{-1} M q_i \\ &= v_j K_\sigma^{-1} M q_j + \sum_{i=1}^{j-1} v_i K_\sigma^{-1} M q_i \end{aligned}$$

Observe that each vector, $K_\sigma^{-1}Mq_i$, in the above summation can be written as a linear combination of the first $i + 1$ Lanczos vectors. Therefore the sum can be written as a linear combination of the first j Lanczos vectors. Consequently

$$v_j = v_j K_\sigma^{-1} M q_j + \sum_{i=1}^{j} \bar{v}_i q_i$$

The M-orthogonalization of v_j against the preceding j Lanczos vectors will purge the component of v_j along each of the q_i's, and therefore the final result will be unaffected by the sum in the last equation. Therefore the next Lanczos vector, q_{j+1}, will be obtained by first computing a preliminary vector $\bar{r}_j$ from the previous vector, q_j,

$$\bar{r}_j = K_\sigma^{-1} M q_j \tag{10.6.9}$$

and M-orthonormalizing it against all the previous Lanczos vectors. Now, in general it may be assumed that this preliminary vector contains components from each of the preceding vectors. Thus

$$\bar{r}_j = r_j + \alpha_j q_j + \beta_j q_{j-1} + \gamma_j q_{j-2} + \cdots \tag{10.6.10}$$

where r_j is the "pure" component of $\bar{r}_j$ orthogonal to all previous Lanczos vectors, and α_j, β_j, γ_j, $\cdots$ are the amplitudes of the previous vectors contained in $\bar{r}_j$. These amplitude coefficients are evaluated from the orthonormality of the Lanczos vectors. Thus, if both sides of (10.6.10) are multiplied by $q_j^T M$, the result is

$$q_j^T M \bar{r}_j = q_j^T M r_j + \alpha_j q_j^T M q_j + \beta_j q_j^T M q_{j-1} + \gamma_j q_j^T M q_{j-2} + \cdots \tag{10.6.11}$$

Here the first term on the right-hand side vanishes by *definition*, and all terms beyond the second vanish similarly due to M-orthogonality. The normalizing definition applied to the second term then reduces (10.6.11) to an expression for the amplitude of q_j along $\bar{r}_j$:

$$\boxed{\alpha_j = q_j^T M \bar{r}_j} \tag{10.6.12}$$

The amplitude of q_{j-1} contained in $\bar{r}_j$ may be found similarly by multiplying (10.6.10) by $q_{j-1}^T M$. In this case all terms except the third vanish by orthogonality, and the coefficient of β_j is unity, so $\beta_j = q_{j-1}^T M \bar{r}_j$. But, using (10.6.9) to eliminate $\bar{r}_j$ this gives $\beta_j = q_{j-1}^T M K_\sigma^{-1} M q_j$ and applying the transpose of (10.6.9) to the q_{j-1}^T vector gives

$$\beta_j = \bar{r}_{j-1}^T M q_j \tag{10.6.13}$$

Finally, expanding $\bar{r}_{j-1}$ in terms of its pure component, r_{j-1}, and the preceding Lanczos vectors, as in (10.6.10), the transpose of (10.6.13) becomes

$$\beta_j = q_j^T M r_{j-1} + \alpha_{j-1} q_j^T M q_{j-1} + \beta_{j-1} q_j^T M q_{j-2} + \gamma_{j-1} q_j^T M q_{j-3} + \cdots \tag{10.6.14}$$

It is evident that all terms except the first vanish on the right-hand side. Now q_j is the vector obtained by normalizing r_{j-1}, i.e.,

$$q_j = \frac{1}{\|r_{j-1}\|_M} r_{j-1} \tag{10.6.15}$$

where $\|r_{j-1}\|_M = (r_{j-1}^T M r_{j-1})^{1/2}$. Using this expression for q_j in (10.6.14), we obtain β_j as $\beta_j = (1/\|r_{j-1}\|_M)\, r_{j-1}^T M r_{j-1}$, or

$$\boxed{\beta_j^2 = r_{j-1}^T M r_{j-1}} \tag{10.6.16}$$

This is an alternative to (10.6.13) for evaluating β_j. In [26] Scott established that computing β_j using (10.6.16) is preferable to (10.6.13) and numerical experiments also confirm his results. Using (10.6.16) ensures that the Lanczos vectors are properly normalized even if $q_{j+1}^T M q_{j-1}$ is not exactly zero.

Continuing in the same way, the amplitude of q_{j-2} contained in $\bar{r}_j$ is found to be

$$\gamma_j = q_{j-2}^T M \bar{r}_j \tag{10.6.17}$$

Following the procedure used to derive (10.6.14), this leads to

$$\gamma_j = q_j^T M r_{j-2} + \alpha_{j-2} q_j^T M q_{j-2} + \beta_{j-2} q_j^T M q_{j-3} + \gamma_{j-2} q_j^T M q_{j-4} + \cdots \tag{10.6.18}$$

But, using the normalizing relationship equivalent to (10.6.15), $r_{j-2} = \beta_{j-1} q_{j-1}$. Hence, when this is substituted into (10.6.18) all terms on the right-hand side vanish, with the result that $\gamma_j = 0$. A corresponding procedure could be used to demonstrate that all further terms in the expansion for $\bar{r}_j$, (10.6.10), vanish; in other words, ***the orthogonalization procedure used in generating each Lanczos vector need be applied only to the previous two vectors.***

A summary of the Lanczos algorithm is presented in Table 10.6.1.

TABLE 10.6.1. The Lanczos Algorithm

Given an arbitrary vector r_0 then:

1. Set
   a. $q_0 = 0$
   b. $\beta_1 = (r_0^T M r_0)^{1/2}$
   c. $q_1 = \dfrac{r_0}{\beta_1}$
   d. $p_1 = M q_1$
2. For $j = 1, 2, \ldots,$ repeat:
   a. $\bar{r}_j = K_\sigma^{-1} p_j$
   b. $\hat{r}_j = \bar{r}_j - q_{j-1}\beta_j$
   c. $\alpha_j = q_j^T M \hat{r}_j = p_j^T \hat{r}_j$
   d. $r_j = \hat{r}_j - q_j \alpha_j$
   e. $\bar{p}_j = M r_j$
   f. $\beta_{j+1} = (r_j^T M r_j)^{1/2} = (\bar{p}_j^T r_j)^{1/2}$
   g. if enough vectors, then terminate the loop
   h. $q_{j+1} = \dfrac{1}{\beta_{j+1}} r_j$
   i. $p_{j+1} = \dfrac{1}{\beta_{j+1}} \bar{p}_j$

The above process may be started from a random vector, r_0, with $q_0 = 0$ and $\beta_1 = (r_0^T M r_0)^{1/2}$. At a typical step, j, the Lanczos algorithm computes α_j, β_{j+1}, and q_{j+1}, in order. In addition to the storage needs of K_σ and M, the algorithm requires storage for five vectors of length n; one for each of the vectors, q_{j-1}, q_j, Mr_j, p_j, and r_j. The total cost for one step of the algorithm involves a multiply with M, the solution of a system of equations with K_σ as the coefficient matrix, two inner products and four products of a scalar by a vector.

10.6.6 Reduction to Tridiagonal Form

Using the results of the previous section, (10.6.10) can be rewritten as the three-term relation

$$\boxed{r_j = \beta_{j+1} q_{j+1} = K_\sigma^{-1} M q_j - q_j \alpha_j - q_{j-1} \beta_j} \tag{10.6.19}$$

where $\alpha_j = q_j^T M K_\sigma^{-1} M q_j$ and r_j is normalized with respect to the mass matrix to obtain q_{j+1} with normalizing factor $\beta_{j+1} = (r_j^T M r_j)^{1/2}$. After m Lanczos steps all the quantities obtained from equation (10.6.19) can be arranged in a global matrix form

$$\left[K_\sigma^{-1} M\right]\left[Q_m\right] - \left[Q_m\right]\left[T_m\right] = \left[0 \;\vdots\; 0 \;\vdots\; \cdots \;\vdots\; 0 \;\vdots\; r_m\right] = r_m e_m^T \tag{10.6.20}$$

Here $e_m^T = \langle 0,0,\cdots,0,1\rangle$, Q_m is an $n \times m$ matrix with columns q_i, $i = 1,2,\cdots,m$, and T_m is a ***tridiagonal matrix*** of the form

$$T_m = \begin{bmatrix} \alpha_1 & \beta_2 & & & & \\ \beta_2 & \alpha_2 & \beta_3 & & & \\ & \beta_3 & \cdot & \cdot & & \\ & & \cdot & \cdot & \cdot & \\ & & & \cdot & \cdot & \cdot \\ & & & & \cdot & \cdot \; \beta_m \\ & & & & & \beta_m \; \alpha_m \end{bmatrix} \tag{10.6.21}$$

The orthogonality property of the Lanczos vectors, $Q_m^T M Q_m = I_m$, where I_m is the $m \times m$ identity matrix, can be used in (10.6.20) to obtain

$$\boxed{Q_m^T M K_\sigma^{-1} M Q_m = T_m} \tag{10.6.22}$$

Choosing the set of Lanczos vectors, Q_m, for the trial vectors, the Rayleigh-Ritz procedure can be used to obtain the best approximation to the eigenvectors of (10.6.6). The approximating Ritz vectors will then be of the form

$$y_i^{(m)} = Q_m s_i^{(m)}, \quad i = 1, \ldots, m \tag{10.6.23}$$

When a residual vector, $\{K_\sigma^{-1}My_i^{(m)} - \theta_i^{(m)}y_i^{(m)}\}$, associated with the pair $\{\theta_i^{(m)}, y_i^{(m)}\}$ is M-orthogonal to the set of Lanczos vectors, then $\{\theta_i^{(m)}, y_i^{(m)}\}$ is a Ritz pair. Accordingly

$$Q_m^T M\{K_\sigma^{-1}My_i^{(m)} - \theta_i^{(m)}y_i^{(m)}\} = 0$$

Using the relation between the Ritz vector, $y_i^{(m)}$ and the Lanczos vectors, given in (10.6.23), together with the orthonormality condition of the q's, and the tridiagonal properties of the Lanczos vectors, (10.6.22), the above equation reduces to the tridiagonal eigenproblem

$$\boxed{T_m s_i^{(m)} - \theta_i^{(m)}s_i^{(m)} = 0} \tag{10.6.24}$$

Thus $\{\theta_i^{(m)}, s_i^{(m)}\}$ is an eigenpair of the tridiagonal matrix, T_m. As the total number of Lanczos vectors increases, i.e., as we take more Lanczos steps, the size of the tridiagonal matrix increases and the eigenvalues of T_m converge to the eigenvalues of the transformed problem (10.6.6), $1/(\lambda_i - \sigma)$. When $m = n$, the order of K, then $\theta_i^{(n)} = 1/(\lambda_i - \sigma)$ for all i, but we hope to stop long before $m = n$. The steps enumerated in Step 2 of Table 10.6.1 are repeated until the Ritz pairs $\{\theta_i^{(m)}, y_i^{(m)}\}$ have sufficiently converged to the desired eigenpairs.

Example 2

$$K = \frac{1}{80}\begin{bmatrix} 39 & -9 & 21 & -11 \\ -9 & 39 & -11 & 21 \\ 21 & -11 & 39 & -9 \\ -11 & 21 & -9 & 39 \end{bmatrix}$$

For this example we assume the mass matrix is the identity. Then the eigenvalue matrix for this problem is $\Lambda = \text{diag}\,(\frac{1}{5}, \frac{1}{4}, \frac{1}{2}, 1)$. The Lanczos algorithm presented here requires the solution of a linear system of equations with K. We therefore give K^{-1} explicitly for convenience. Accordingly,

$$K^{-1} = \frac{1}{2}\begin{bmatrix} 6 & 0 & -3 & 1 \\ 0 & 6 & 1 & -3 \\ -3 & 1 & 6 & 0 \\ 1 & -3 & 0 & 6 \end{bmatrix}$$

Choosing a starting vector $r_0 = \langle 1, 0, 0, 0\rangle$, then

Step 1:

$$\beta_1 = \|r_0\| = 1, \qquad q_1 = \frac{r_0}{\beta_1} = \begin{Bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{Bmatrix}, \qquad K^{-1}q_1 = \frac{1}{2}\begin{Bmatrix} 6 \\ 0 \\ -3 \\ 1 \end{Bmatrix}$$

$$\alpha_1 = \boldsymbol{q}_1^T \boldsymbol{K}^{-1} \boldsymbol{q}_1 = 3, \qquad \boldsymbol{r}_1 = \boldsymbol{K}^{-1} \boldsymbol{q}_1 - \alpha_1 \boldsymbol{q}_1 = \frac{1}{2} \begin{Bmatrix} 0 \\ 0 \\ -3 \\ 1 \end{Bmatrix}, \qquad \boldsymbol{T}_1 = [3]$$

Step 2:

$$\beta_2 = \|\boldsymbol{r}_1\| = \frac{\sqrt{10}}{2}, \qquad \boldsymbol{q}_2 = \frac{\boldsymbol{r}_1}{\beta_2} = \frac{1}{\sqrt{10}} \begin{Bmatrix} 0 \\ 0 \\ -3 \\ 1 \end{Bmatrix}, \qquad \boldsymbol{K}^{-1} \boldsymbol{q}_2 = \frac{1}{\sqrt{10}} \begin{Bmatrix} 5 \\ -3 \\ -9 \\ 3 \end{Bmatrix}$$

$$\alpha_2 = \boldsymbol{q}_2^T \boldsymbol{K}^{-1} \boldsymbol{q}_2 = 3, \qquad \boldsymbol{r}_2 = \boldsymbol{K}^{-1} \boldsymbol{q}_2 - \alpha_2 \boldsymbol{q}_2 - \beta_2 \boldsymbol{q}_1 = \frac{1}{\sqrt{10}} \begin{Bmatrix} 0 \\ -3 \\ 0 \\ 0 \end{Bmatrix}$$

$$\boldsymbol{T}_2 = \begin{bmatrix} 3 & \frac{\sqrt{10}}{2} \\ \frac{\sqrt{10}}{2} & 3 \end{bmatrix}$$

Step 3:

$$\beta_3 = \|\boldsymbol{r}_2\| = \frac{3}{\sqrt{10}}, \qquad \boldsymbol{q}_3 = \frac{\boldsymbol{r}_2}{\beta_3} = \begin{Bmatrix} 0 \\ -1 \\ 0 \\ 0 \end{Bmatrix}, \qquad \boldsymbol{K}^{-1} \boldsymbol{q}_3 = \frac{1}{2} \begin{Bmatrix} 0 \\ -6 \\ -1 \\ 3 \end{Bmatrix}$$

$$\alpha_3 = \boldsymbol{q}_3^T \boldsymbol{K}^{-1} \boldsymbol{q}_3 = 3, \qquad \boldsymbol{r}_3 = \boldsymbol{K}^{-1} \boldsymbol{q}_3 - \alpha_3 \boldsymbol{q}_3 - \beta_3 \boldsymbol{q}_2 = \frac{1}{5} \begin{Bmatrix} 0 \\ 0 \\ 2 \\ 6 \end{Bmatrix}$$

$$\boldsymbol{T}_3 = \begin{bmatrix} 3 & \frac{\sqrt{10}}{2} & 0 \\ \frac{\sqrt{10}}{2} & 3 & \frac{3}{\sqrt{10}} \\ 0 & \frac{3}{\sqrt{10}} & 3 \end{bmatrix}$$

Step 4:

$$\beta_4 = \|\boldsymbol{r}_3\| = \frac{4}{\sqrt{10}}, \qquad \boldsymbol{q}_4 = \frac{\boldsymbol{r}_3}{\beta_4} = \frac{1}{\sqrt{10}} \begin{Bmatrix} 0 \\ 0 \\ 1 \\ 3 \end{Bmatrix}, \qquad \boldsymbol{K}^{-1} \boldsymbol{q}_4 = \frac{1}{\sqrt{10}} \begin{Bmatrix} 0 \\ -4 \\ 3 \\ 9 \end{Bmatrix}$$

$$\alpha_4 = q_4^T K^{-1} q_4 = 3, \qquad r_4 = K^{-1} q_4 - \alpha_4 q_4 - \beta_4 q_3 = \begin{Bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{Bmatrix}$$

$$T_4 = \begin{bmatrix} 3 & \frac{\sqrt{10}}{2} & 0 & 0 \\ \frac{\sqrt{10}}{2} & 3 & \frac{3}{\sqrt{10}} & 0 \\ 0 & \frac{3}{\sqrt{10}} & 3 & \frac{4}{\sqrt{10}} \\ 0 & 0 & \frac{4}{\sqrt{10}} & 3 \end{bmatrix}$$

The eigenvalues of the tridiagonal matrices converge to the *inverses* of the eigenvalues of K in this example. This can be demonstrated by computing the eigenvalues of the tridiagonal at each step as shown in Table 10.6.2

TABLE 10.6.2 Convergence of the Ritz Values for the Small 4 × 4 Example

j	Eigenvalues of T_j
1	3.0000
2	1.4189, 4.5811
3	1.1561, 3.0000, 4.8439
4	1.0000, 2.0000, 4.0000, 5.0000

In Figure 10.6.1 we plot the Ritz values for a larger example. Notice the early convergence at both ends of the spectrum.

10.6.7 Convergence Criterion for Eigenvalues

The eigenvalues, $\theta_i^{(j)}$, $i = 1, \ldots, j$, of the tridiagonal T_j are the Rayleigh-Ritz approximations to eigenvalues of (10.6.6). As j increases these Ritz values get closer to the eigenvalues they are approximating. Often users wait until the change in the computed quantities is within a specified tolerance; that is

$$\frac{|\theta^{(j-1)} - \theta^{(j)}|}{|\theta^{(j)}|} \leq \text{tol} \tag{10.6.25}$$

Here we demonstrate the weakness of such convergence tests. Suppose one is performing inverse iteration with a specified shift, σ, in an interval of interest $[\sigma_1, \sigma_2]$; i.e.,

$$\sigma = \gamma\sigma_1 + (1 - \gamma)\sigma_2$$

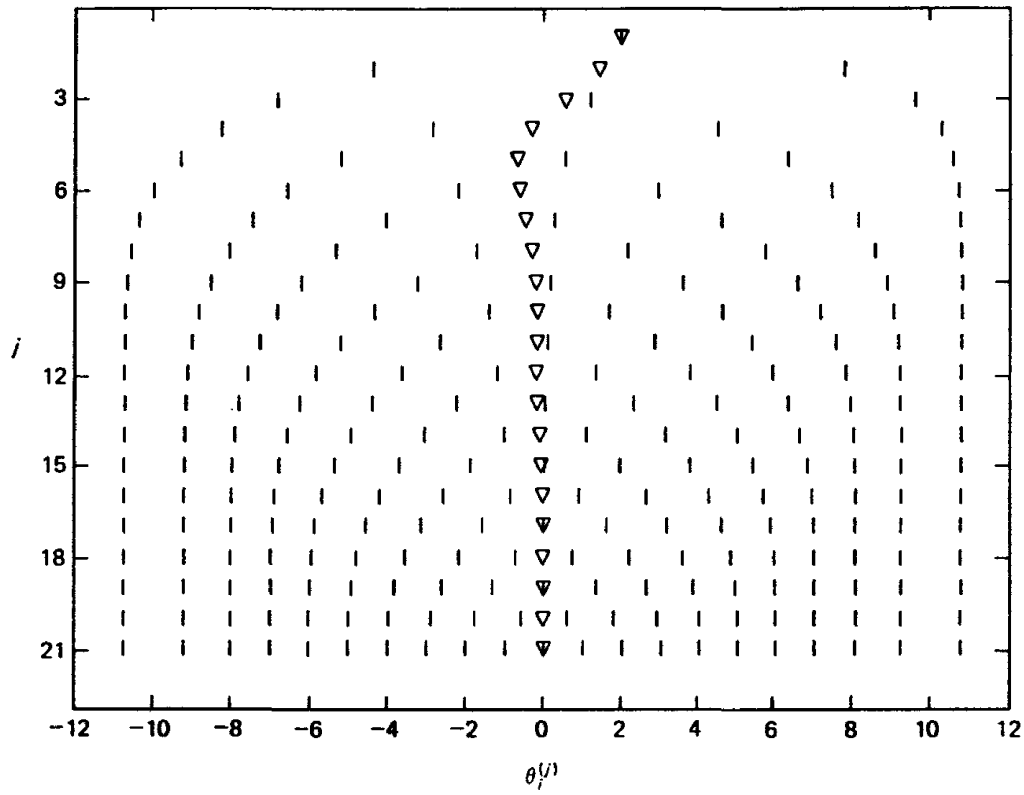


Figure 10.6.1 A typical pattern for the progress of the eigenvalues of the tridiagonal matrix, T_j. The symbol ∇ indicates the last diagonal entry of T_j.

where γ is a positive number less than 1. Further, assume that the problem has eigenvalues, λ_a and λ_b outside this interval with $\lambda_a < \sigma_1$ and $\lambda_b > \sigma_2$. Then the corresponding eigenvalues of the transformed problem, (10.6.6), are $\nu_a = 1/(\lambda_a - \sigma)$ and $\nu_b = 1/(\lambda_b - \sigma)$, with $\nu_a < 0$ and $\nu_b > 0$. Now, let us perform a step of the inverse iteration method with an unfortunate starting vector, $x^{(1)} = z_a \sin \psi + z_b \cos \psi$. Here, z_a and z_b are eigenvectors corresponding to λ_a and λ_b, respectively. We assume that the eigenvectors are normalized with respect to the mass matrix. Then $x^{(1)}$ will also be normalized. The Ritz value due to $x^{(1)}$ is

$$\begin{aligned}\theta^{(1)} &= x^{(1)T} K x^{(1)} \\ &= \lambda_a \sin^2 \psi + \lambda_b \cos^2 \psi\end{aligned}$$

After one step of inverse iteration the improved vector is

$$x^{(2)} = \frac{1}{[\nu_a^2 \sin^2 \psi + \nu_b^2 \cos^2 \psi]^{1/2}} (\nu_a z_a \sin\psi + \nu_b z_b \cos\psi)$$

and the Ritz value associated with $x^{(2)}$ is

$$\theta^{(2)} = \frac{1}{[\nu_a^2 \sin^2 \psi + \nu_b^2 \cos^2 \psi]^{1/2}} (\nu_a^2 \lambda_a \sin^2 \psi + \nu_b^2 \lambda_b \cos^2 \psi)$$

Now if $\lambda_a = \sigma - \tau$ and $\lambda_b = \sigma + \tau$, in which case $\nu_a = -1/\tau$ and $\nu_b = 1/\tau$, then

$$\theta^{(1)} = \theta^{(2)} = (\sigma - \tau) \sin^2 \psi + (\sigma + \tau) \cos^2 \psi = \sigma + \tau \cos 2\psi$$

These θ's, referred to as "ghost" eigenvalues in [25], satisfy (10.6.25) and therefore would be accepted as eigenvalues. Further, an appropriate choice for ψ can result in a Ritz value that is in the interval of interest. This behavior, although rare, has been observed in certain implementations of the subspace iteration method [25] and reflects a serious difficulty. We refer to this as ***misconvergence*** in [23] and, as we observed therein, it cannot be detected by a natural criterion such as (10.6.25). A rigorous inexpensive termination criterion can be obtained using the residual error bound given later.

Consider a pair $\{\theta, y\}$, which approximates an eigenpair $\{\nu, z\}$, of (10.6.6), where ν is the closest eigenvalue of (10.6.6) to θ. For simplicity we consider only normalized Ritz vectors, $\|y\|_M = 1$ where $\|y\|_M = (y^T M y)^{1/2}$. The residual vector associated with this approximation is given by

$$r = K_\sigma^{-1} M y - \theta y \tag{10.6.26}$$

The norm of the residual vector can then be used to assess the accuracy of θ through the inequality

$$\boxed{|\nu - \theta| \leq \|r\|_M} \tag{10.6.27}$$

For the proof and an in-depth study see [21]. This result allows us to predict the accuracy of any candidate eigenpair of a matrix.

We use the residual vector associated with the Ritz pair $\{\theta_i^{(m)}, y_i^{(m)}\}$ given by (10.6.23) and (10.6.24) to check for their convergence. The tridiagonal property of the Lanczos algorithm, (10.6.20), greatly simplifies the computation of the residual norm. Postmultiplying (10.6.20) by $s_i^{(m)}$ leads to

$$K_\sigma^{-1} M Q_m s_i^{(m)} - Q_m T_m s_i^{(m)} = r_m e_m^T s_i^{(m)} \tag{10.6.28}$$

Since $s_i^{(m)}$ is an eigenvector of T_m (see (10.6.24)), we can replace $T_m s_i^{(m)}$ by $\theta_i^{(m)} s_i^{(m)}$. Further, by definition (10.6.23), $Q_m s_i^{(m)}$ is simply the Ritz vector y_i^m and so (10.6.28) will reduce to

$$K_\sigma^{-1} M y_i^{(m)} - \theta_i^{(m)} y_i^{(m)} = r_m e_m^T s_i^{(m)} \tag{10.6.29}$$

Taking norms,

$$\boxed{\begin{aligned} \|K_\sigma^{-1} M y_i^{(m)} - \theta_i^{(m)} y_i^{(m)}\|_M &= \|r_m e_m^T s_i^{(m)}\|_M \\ &= \|r_m\|_M |e_m^T s_i^{(m)}| \\ &= \beta_{m+1} |\zeta_i| \end{aligned}} \tag{10.6.30}$$

where $\zeta_i = e_m^T s_i^{(m)}$ is the bottom element of $s_i^{(m)}$, the normalized eigenvector of T_m. β_{m+1} is a scalar quantity that is computed in the course of the Lanczos process. The bottom element of the eigenvector of T_m can be obtained at little cost once the

associated eigenvalue is found. Therefore, the residual norm associated with a Ritz pair can be computed as $\rho_{mi} = \beta_{m+1} |\zeta_i|$ and the Ritz value is considered converged once $\rho_{mi} < \text{tol}$. There is no need to compute $y_i^{(m)}$ until it has converged.

Multiple Eigenvalues

In exact arithmetic the simple Lanczos algorithm cannot compute a second copy of a multiple eigenvalue. Suppose λ is an eigenvalue with multiplicity two. Then there is no unique eigenvector associated with λ. In fact one can obtain two vectors, z_1 and z_2, such that any linear combination of these vectors is also an eigenvector of λ. Moreover, it is possible to obtain a linear combination of z_1 and z_2, $\bar{z} = \xi_1 z_1 + \xi_2 z_2$, that is orthogonal to q_1. Then the eigenvector $\bar{z}$ will be orthogonal to all the subsequent Lanczos vectors. Therefore $\bar{z}$ will never be found.

Fortunately, this argument only holds in exact arithmetic. In finite precision, roundoff comes to the rescue. If q_1 is perfectly orthogonal to an eigenvector, $\bar{z}$, then because of roundoff error, q_2 will have a small component along $\bar{z}$. This component, although tiny, will eventually grow such that $\bar{z}$ can be represented by a linear combination of the computed Lanczos vectors. However, the second copy of λ will converge some steps after the first has converged.

10.6.8 Loss of Orthogonality

In a previous section we derived the equations governing the Lanczos algorithm. These relations are only satisfied by quantities obtained in exact arithmetic. In finite precision, however, each computation introduces a small error and therefore each computed quantity will differ from its exact counterpart. Our objective here is to describe the effect of roundoff on the Lanczos process. For this purpose we need to introduce an important quantity that measures the accuracy of the arithmetic. Let ϵ be the smallest number in the computer such that $1 + \epsilon > 1$. It is known as the ***unit roundoff error***.

Although the tridiagonal relation, (10.6.20), is preserved to within roundoff, the M-orthogonality property of the Lanczos vectors completely breaks down after a certain number of steps, depending on ϵ and the distribution of the eigenvalues of (10.6.6). The Lanczos vectors, which are orthogonal in exact arithmetic, not only lose their orthogonality, but may even become linearly dependent. Initially it was believed that loss of orthogonality is due to cancellations that occur each time r_j is evaluated using (10.6.19). This step is simply M-orthogonalization of $K_\sigma^{-1} M q_j$ against q_j and q_{j-1}. Comparing the final vector resulting from this computation to the starting vector, one can obtain a measure of the cancellation that occurs in this step; i.e., the ratio

$$\chi_j^2 = \frac{\|r_j\|_M^2}{\|K_\sigma^{-1} M q_j\|_M^2} = \frac{\beta_{j+1}^2}{\beta_j^2 + \alpha_j^2 + \beta_{j+1}^2} \tag{10.6.31}$$

indicates how much cancellation has occurred. χ_j is the sine of the angle between the vector $K_\sigma^{-1} M q_j$ and the plane containing the vectors q_j and q_{j-1}. When χ_j is zero it indicates that in the jth Lanczos step we are orthogonalizing a vector that is already

in this plane against q_j and q_{j-1}, and therefore complete cancellation occurs. In practice χ_j rarely drops below $\frac{1}{10}$. When χ_j drops to $\frac{1}{100}$, for instance, it indicates that $K_\sigma^{-1}Mq_j$ is nearly parallel to this plane, and therefore at the end of this Lanczos step the computed r_j may not be orthogonal to the plane containing q_j and q_{j-1} to working accuracy. In such cases r_j should be orthogonalized against q_j and q_{j-1} a second time. For a long time it was believed that loss of orthogonality was solely due to this cancellation. Indeed if this were the case, then we would have no option other than a complete reorthogonalization at each step. However, as we soon shall see, a completely different mechanism is at work.

Suppose for the moment the algorithm was executed in exact arithmetic for j steps, except that at some step $k < j$ a small error was introduced into the computation of q_k. The first $k - 1$ Lanczos vectors will be perfectly M-orthonormal, but they will not be orthogonal to all the vectors computed after the kth step. The error introduced at step k will be amplified in the subsequent steps to such an extent that linear independency may also be lost.

Hence, the loss of orthogonality can be viewed as the subsequent amplification of the error introduced after each computation. To analyze the way in which orthogonality deteriorates, we let Q_m denote the computed Lanczos vectors and define the following matrix:

$$H_m = Q_m^T M Q_m \tag{10.6.32}$$

where the i,j-component of H_m is $\eta_{i,j} = q_i^T M q_j$. In exact arithmetic H_m is the identity matrix. The off-diagonals of H_m will depend on ϵ, the unit roundoff error. Further, (10.6.19) will be satisfied by the computed quantities only to within roundoff error. Now, multiply (10.6.19) by $q_i^T M$ and use the above definition to get the approximate relation

$$q_i^T M K_\sigma^{-1} M q_j \cong \alpha_j \eta_{i,j} + \beta_j \eta_{i,j-1} + \beta_{j+1} \eta_{i,j+1} \tag{10.6.33}$$

A similar equation can be obtained when the above procedure is repeated for the ith Lanczos step

$$q_j^T M K_\sigma^{-1} M q_i \cong \alpha_i \eta_{j,i} + \beta_i \eta_{j,i-1} + \beta_{i+1} \eta_{j,i+1} \tag{10.6.34}$$

By symmetry of $MK_\sigma^{-1}M$, the terms on the left-hand side of (10.6.33) and (10.6.34) are equal and therefore can be eliminated by subtraction, resulting in the relation

$$\beta_{j+1} \eta_{j+1,i} \cong \beta_{i+1} \eta_{j,i+1} + (\alpha_i - \alpha_j)\eta_{j,i} + \beta_i \eta_{j,i-1} - \beta_j \eta_{j-1,i} \tag{10.6.35}$$

This recursion holds for $j \geq 2$ and $1 \leq i \leq j - 1$ and starts by assuming that the diagonal of H_m is the identity, $\eta_{j,j} = 1$ for all $j \geq 1$, and the first off-diagonal of H_m is at roundoff level, $\eta_{j,j-1} = \epsilon$ for $j \geq 2$. The above relation is due to Simon [27]. It provides a means of estimating the elements of a column of H_m from the elements of T_m and the elements in the previous columns of H_m. This recursion can be restated in the vector form

$$\boxed{\beta_{j+1} h_{j+1} \cong T_{j-1} h_j - \alpha_j h_j - \beta_j h_{j-1}} \tag{10.6.36}$$

where $\boldsymbol{h}_{j-1}$, $\boldsymbol{h}_j$ and $\boldsymbol{h}_{j+1}$ are vectors of length $j-1$ containing the top $j-1$ elements of the $(j-1)$st, jth, and $(j+1)$st columns of $(\boldsymbol{H}_m - \boldsymbol{I}_m)$. Here, the bottom element of $\boldsymbol{h}_{j-1}$ is zero. Then, all the terms in $\boldsymbol{h}_j$ depend on ϵ. Taking norms we can show

$$\begin{aligned}\beta_{j+1}\|\boldsymbol{h}_{j+1}\| &\leq (\|\boldsymbol{T}_{j-1}\| + |\alpha_j|)\|\boldsymbol{h}_j\| + \beta_j\|\boldsymbol{h}_{j-1}\| \\ &\leq 2\|\boldsymbol{T}_j\|\max(\|\boldsymbol{h}_j\|, \|\boldsymbol{h}_{j-1}\|) \end{aligned} \tag{10.6.37}$$

This result shows that the level of nonorthogonality can grow by at most a factor of $2\|\boldsymbol{T}_j\|/\beta_{j+1}$ after each step. A drop in the value of β_{j+1} can result in a sudden loss of orthogonality but this rarely occurs.

An alternative characterization of the pattern in which orthogonality is lost was presented by Paige [18–20]. Instead of examining the vector $\boldsymbol{h}_{j+1} = \boldsymbol{Q}_j^T\boldsymbol{M}\boldsymbol{q}_{j+1}$, he looked at a linear combination of the components in this vector. To be more specific, he examined the inner product between each Ritz vector $\boldsymbol{y}_i^{(j)}$ given by (10.6.23) and $\boldsymbol{q}_{j+1}$. That is

$$\begin{aligned}\boldsymbol{y}_i^{(j)\,T}\boldsymbol{M}\boldsymbol{q}_{j+1} &= \boldsymbol{s}_i^{(j)\,T}\boldsymbol{Q}_j^T\boldsymbol{M}\boldsymbol{q}_{j+1} \\ &= \boldsymbol{s}_i^{(j)\,T}\boldsymbol{h}_{j+1} \end{aligned} \tag{10.6.38}$$

In exact arithmetic this value is zero. However, in his work Paige showed that

$$\boldsymbol{y}_i^{(j)\,T}\boldsymbol{M}\boldsymbol{q}_{j+1} = \frac{\gamma_{ji}\,\epsilon\|\boldsymbol{T}_j\|}{\beta_{j+1}|\zeta_i|} \tag{10.6.39}$$

Recall that ζ_i is the bottom element of $\boldsymbol{s}_i^{(j)}$, the ith eigenvector of $\boldsymbol{T}_j$. γ_{ji} is a scalar quantity usually close to unity. We omit the derivation of this result and refer the interested reader to [21]. Note that Paige's result also shows that a sudden drop in β_{j+1} can result in a severe loss of orthogonality. Moreover, recall the quantity in the denominator is also a measure of the convergence of the Ritz value $\theta_i^{(j)}$ (see (10.6.30)). The only way the left side of (10.6.39) can rise up to values like 0.1 is for $\rho_{ji}\,(=\beta_{j+1}|\zeta_i|)$ to drop down to $10\gamma_{ji}\epsilon\|\boldsymbol{T}_j\|$, so

> ***Loss of orthogonality $\Rightarrow$ convergence of a Ritz value***

When only a single Ritz value converges at step j, then $\boldsymbol{q}_{j+1}$ loses orthogonality by tilting toward the converged Ritz vector, which in turn is a linear combination of the previous Lanczos vectors. When more than one Ritz vector converges simultaneously, the picture is more complicated. In this case $\boldsymbol{q}_{j+1}$ tilts toward a linear combination of these vectors.

Return of Banished Ritz Vectors

In theory, if two successive Lanczos vectors, $\boldsymbol{q}_{j-1}$ and $\boldsymbol{q}_j$, are orthogonal to an eigenvector, $\boldsymbol{z}$, then all the subsequent Lanczos vectors will also be orthogonal to $\boldsymbol{z}$. In practice, however, a converged Ritz vector, $\boldsymbol{y}_i$, will not remain orthogonal to all the subsequent Lanczos vectors. This is a consequence of the same mechanism by which

multiple eigenvalues are found. Roundoff errors will add to each Lanczos vector a small component of y_i, which will grow eventually to such a magnitude that orthogonality to y_i is lost. This phenomenon can also be observed in the inverse iteration method; i.e., orthogonalizing the starting vector against the first eigenvector does not guarantee convergence of the iteration vectors to the next eigenvector.

The state of orthogonality between a converged Ritz vector, y_i, and the current Lanczos vector, q_j, can be measured by the component of y_i along q_j. We define $\tau_j = y_i^T M q_j$. Then multiplying (10.6.19) by $y_i^T M$ and considering the effect of round-off on (10.6.19) yields

$$\beta_{j+1} y_i^T M q_{j+1} \cong y_i^T M K_\sigma^{-1} M q_j - \alpha_j y_i^T M q_j - \beta_j y_i^T M q_{j-1}$$

Using the fact that y_i is a converged Ritz value, i.e., $K_\sigma^{-1} M y_i = \theta_i y_i$, together with the definition of τ_j we obtain

$$\boxed{\tau_{j+1} \cong \frac{(\theta_i - \alpha_j)\tau_j - \beta_j \tau_{j-1}}{\beta_{j+1}}} \tag{10.6.40}$$

This recurrence can be updated for each converged Ritz value, θ_i. The magnitude of τ_j can be used as an indicator for loss of orthogonality against a converged Ritz vector. The growth of the components of the Lanczos vectors along a converged Ritz vector is referred to as the ***return of a banished Ritz vector***.

10.6.9 Restoring Orthogonality

In this section we look at a number of preventive measures that we can adopt to maintain a certain level of orthogonality. Lanczos [14] was aware of the effects of round-off on the algorithm when he presented his work. He proposed that the newly computed Lanczos vector, q_{j+1}, be explicitly orthogonalized against all the preceding vectors at the end of each step j. We will refer to this technique as the "full reorthogonalization" method. This scheme is adopted in [12, 29]. With this procedure the Lanczos vectors will meet the stringent requirement

$$|q_i^T M q_j| < n\epsilon, \quad i \neq j \tag{10.6.41}$$

Although this scheme increases the overall cost of an eigenvalue computation, for short Lanczos runs (when the number of Lanczos steps is less than the half-bandwidth of K) the increase in cost is small compared to the cost of solving a system of equations with K_σ. For longer runs the cost of a full reorthogonalization step will begin to dominate the cost of a Lanczos step, although vector computers will delay this effect.

The orthogonality condition of (10.6.41) can be replaced by a more relaxed condition,

$$|q_i^T M q_j| < \sqrt{n\epsilon}, \quad i \neq j \tag{10.6.42}$$

We refer to this as the ***semiorthogonality condition*** and we refer to procedures that adopt the weaker condition as ***selective orthogonalization methods***. Imposing the

more relaxed condition can result in considerable reduction in the number of operations in the reorthogonalization step and semiorthogonality is sufficient to make T_j exact to working precision.

We consider two different reorthogonalization schemes that adopt the more relaxed condition of (10.6.42).

A. ***Orthogonalization against Ritz vectors:*** This procedure is a consequence of the result of Paige [18–20]. As soon as a Ritz value converges, its corresponding Ritz vector is computed and the component of this vector along q_{j+1} is purged. Further, for each converged Ritz value, the three-term recurrence for τ_j is updated and whenever τ_j becomes greater than $\sqrt{\epsilon}$ in absolute value, it signals that the component of the corresponding eigenvector has grown too much. So the new Lanczos vector is orthogonalized against this known eigenvector [22, 25].

B. ***Orthogonalization against previous Lanczos vectors***: This scheme can be based on either of the techniques given in [13,27]. In [27] the vector h_{j+1} is updated using (10.6.36) and the magnitudes of its elements are monitored. Whenever the ith element of h_{j+1} is greater than $\sqrt{\epsilon}$ then semiorthogonality is lost between q_{j+1} and q_i. At this step the appropriate Lanczos vectors are brought in from secondary storage and their components along q_{j+1} are removed. This scheme is also adopted in [17].

Both schemes indicate loss of orthogonality at about the same step. The first method performs orthogonalization against fewer vectors and therefore costs less. However, alone it has some shortcomings. The result in (10.6.39) gives an accurate picture at the early stages of a Lanczos run when one or two Ritz values converge. When for a number of steps, a few Ritz values converge at the same time, then a gradual loss of semiorthogonality may occur. This appears most often in a Lanczos run with spectral transformation. For this reason we do not use scheme A on its own. However, for short Lanczos runs where only a very few (less than 10) eigenpairs are wanted, scheme A is very effective.

We advocate a combination of the above two schemes. Whenever possible we orthogonalize against a Ritz vector (scheme A) because of lower costs. We update h_{j+1} using (10.6.36) and monitor its elements. If any of the elements of h_{j+1} is greater than $\sqrt{\epsilon}$, then semiorthogonality may have been lost. This is the only procedure we use to determine loss of semiorthogonality, which can occur in two possible ways:

i. Convergence of a Ritz value.
ii. Growth of components of computed eigenvectors along the Lanczos vector.

We take different actions for each of these. If the observed orthogonality loss is due to (i), then we perform a step of scheme B. The Lanczos vectors are brought in from secondary storage and the newly computed Lanczos vector, r_j, is orthogonalized against them. Further, to minimize data transfer from secondary storage, we also compute some of the converged Ritz vectors at the same time. Therefore the computation of newly converged Ritz vectors is delayed until the Lanczos vectors are brought back for reorthogonalization. We should note that when (i) occurs, both methods A and B would require recalling the Lanczos vectors from secondary storage

but for different reasons: the first for computing Ritz vectors and the second for reorthogonalization. We perform both. Our method would clearly require more operations than scheme A but has the following two advantages:

a. An eigenvector is computed in a few steps after the convergence of its eigenvalue and therefore has more than half correct digits.
b. The gradual loss of orthogonality just mentioned will not occur with the combined procedure.

However, if loss of orthogonality is due to (ii), then we can use scheme A, i.e., orthogonalize against a computed Ritz vector. For this reason, the τ recurrence, (10.6.40), is also monitored to establish which Ritz vector, y_k, has contaminated q_{j+1}.

The orthogonalization of q_{j+1} against y_k alters the state of orthogonality among the Lanczos vectors. The modified $\bar{q}_{j+1}$ is given by

$$\bar{q}_{j+1} = q_{j+1} - \xi_k y_k$$

where $\xi_k = y_k^T M q_{j+1}$. Let $\bar{h}_{j+1} = Q_j^T M \bar{q}_{j+1}$. Then

$$\begin{aligned} \bar{h}_{j+1} &= Q_j^T M (q_{j+1} - \xi_k y_k) \\ &= h_{j+1} - \xi_k s_k \end{aligned} \tag{10.6.43}$$

An approximation to ξ_k can be obtained via

$$\xi_k = y_k^T M q_{j+1} = s_k^T Q_j^T M q_{j+1} = s_k^T h_{j+1}$$

If the approximate result for ξ_k is used then (10.6.43) reduces to the orthogonalization of h_{j+1} against s_k. This simple step is used to modify h_{j+1} to reflect the changes caused by orthogonalizing q_{j+1} against a Ritz vector. The flowchart in Fig. 10.6.5 (see p. 615) gives a global view of our implementation of the selective orthogonalization algorithm.

When it becomes necessary to restore semiorthogonality, the computed vector q_{j+1} must be orthogonalized against some linear combination of the previous Lanczos vectors. q_j and q_{j-1} remain unchanged at the end of this step. But, at the next step, q_j appears again in the computation of q_{j+2}. If no action is taken, then q_{j+2} will be contaminated by q_j and the reorthogonalization efforts of the previous step would be wasted. Therefore a second reorthogonalization step must be performed. If the Lanczos vectors or the converged Ritz vectors reside in secondary storage, they must be retrieved in two successive steps.

Alternatively, at the same time the orthogonality of q_{j+1} is being restored, we can also perform similar modifications on q_j. Then at the end of the next step, no reorthogonalization of q_{j+2} will be necessary. The number of operations for this scheme is the same as that of the scheme above, but vectors are retrieved only once and therefore the I/O overhead is halved.

10.6.10 LANSEL Package

In this section we describe all the ingredients that go into the LANSEL eigenpackage and provide sufficient detail for installing LANSEL into a finite element program. The

steps in each subroutine are described in sequence followed by the listing. Higher level routines are described first.

User-Supplied Subroutines

The only interface between the algorithm and the eigenproblem is through two user-supplied subroutines. In general, the structures of the matrices K and M vary greatly from problem to problem. No single subroutine can be designed to take advantage of the special structure of K and M for all problems. Furthermore, the characteristics of a given equation solver depend strongly on the computing environment. Only the user is aware of the properties of his or her matrices and the computer system that is being used. Therefore, the job of solving the linear system of equations $K_\sigma \bar{r}_j = p_j$ and multiplication of a vector by the mass matrix, $\bar{p}_j = Mr_j$ is relegated to the user. This has the added advantage that a shifting strategy may be implemented without modifying any part of the routines presented here. The two routines that connect K and M with our program must take the form

1. SUBROUTINE OPK (X, Y, N). This solves the linear system of equations $K_\sigma y = x$. N is the length of the vectors X and Y.
2. SUBROUTINE OPM (X, Y, N). This forms the matrix-vector product $y = Mx$. N is the length of the vectors X and Y.

A third subroutine must also be provided for the management of the Lanczos vectors. From time to time the program calls the subroutine STORE. The subroutine statement for STORE is

SUBROUTINE STORE (V, N, J, ISW)

When ISW = 1 the routine must store the column vector V of length N and identifier J in secondary storage. When ISW = 2, a vector V with identifier J that is in secondary storage must be fetched. This way the user can take full advantage of any data management system that is available.

TABLE 10.6.3 List of Vector Operation Routines used by LANSEL

Name	Description
IDAMAX	Finds the index of the element of a vector with maximum absolute value; $i = \arg(\max_{i=1,n} \lvert y_i \rvert)$.
DAXPY	Computes the product of a scalar, a, by a vector, x, adding the result to a vector, y; $y = ax + y$.
DCOPY	Copies a vector, x, to a vector y; $y = x$.
DDOT	Computes the Euclidean inner product of two vectors, x and y; dot $= x^T y$.
DSCAL	Scales the elements of a vector, y, by a scalar factor, a; $y = ay$.
DZERO	Resets the elements of a vector, y, to zero; $y = 0$.

LANSEL also makes use of subroutines for performing basic vector operations, such as addition of two vectors. Some of these routines may be found in [10]. Many computers have libraries of carefully written assembly language implementations of these subroutines, which are much faster. In Table 10.6.3 we give a list of vector operations used by LANSEL. Alternatively, the user may wish to write his or her own subroutines to perform these tasks.

Project 1. Write a program to set up and solve the 4 × 4 example problem of Sec. 10.6.6. The program should employ the LANSEL package described herein. You will have to provide "user-supplied subroutines" that are general enough to handle the data of the 4 × 4 problem. Print out all intermediate calculations and compare with the results presented in Sec. 10.6.6.

Project 2. Modify program DLEARN (see Chapter 11) to perform eigenvalue/eigenvector calculations by interfacing it with the LANSEL package. DLEARN contains several utility routines (e.g., equation-solving routines, a variety of matrix-vector manipulation routines, etc.) which can be helpful in developing the "user-supplied subroutines" for the LANSEL package. You will also need to write input and output routines for the computed eigenvalues and eigenvectors. (This project requires a fairly comprehensive knowledge of the DLEARN program.)

Subroutine LANDRV

LANDRV is the driver for the main subroutine LANSEL. Table 10.6.4 gives a brief description of the list of parameters for LANDRV.

TABLE 10.6.4 Description of the Parameters for LANDRV

Name	Type	Dimension	Description
N	I	scalar	The dimension of the eigenproblem. K and M are N×N matrices.
LANMAX	I	scalar	Upper limit to the number of Lanczos steps. LANMAX must not exceed N.
MAXPRS	I	scalar	Upper limit to the number of wanted eigenpairs. MAXPRS must not exceed LANMAX.
ENDL	F	scalar	ENDL = $\nu_L = 1/(\lambda_L - \sigma)$, where λ_L is the left end of the interval containing the wanted eigenvalues (see Fig. 10.6.2).
ENDR	F	scalar	ENDR = $\nu_R = 1/(\lambda_R - \sigma)$, where λ_R is the right end of the interval containing the wanted eigenvalues (see Fig. 10.6.2).
NW	I	scalar	Length of the work array W. NW must be at least 6×N + 2×MAXPRS + 10×LANMAX. A larger NW can result in faster computation time. NW need not be greater than 6×N + 2×MAXPRS + LANMAX×(6 + LANMAX).
W	F	NW	Work array of length NW. The first N words of W hold a user-supplied starting vector.

Name	Type	Dimension	Description
IW	I	MAXPRS	Work array of length MAXPRS.
EIG	F	MAXPRS	EIG will hold the converged Ritz values on return. The Ritz values are in the order they were computed.
Y	F	MAXPRS×N	Y will hold the converged Ritz vectors on return. Y is dimensioned as a two-dimensional array Y(N,MAXPRS) and the Ith column of Y holds the Ritz vector corresponding to EIG(I).
IERR	I	scalar	IERR is an error flag. A successful execution is indicated when IERR = 0. For a list of error flags see Table 10.6.5.
NEIG	I	scalar	The number of computed eigenpairs.

LANDRV acts as an interface between the user and LANSEL. It performs the following tasks before calling LANSEL:

i. Checks the control parameters for possible error.
ii. Allocates the working memory for LANSEL.
iii. If user does not supply a starting vector in W then LANDRV supplies a random vector.
iv. Performs the first step of the Lanczos algorithm by calling STPONE.

If an error is detected, IERR is reset and returned as soon as the checking of the control parameters is complete. Each bit of the integer IERR is used as an error flag. IERR is set to zero when entering LANDRV. The Ith bit of IERR can then be reset to 1 using the command IERR = IERR + 2**I. For example the third bit is set to 1 by adding 8 to IERR. This way all the errors in the control parameters can be detected at the same time. Table 10.6.5 gives a description of possible errors indicated by IERR.

TABLE 10.6.5 List of Errors Indicated by IERR

Bit	Indicates
1	N < 0
2	LANMAX < 0
3	ENDR < ENDL
4	MAXPRS < 0
5	MAXPRS > LANMAX
6	LANMAX > N
7	NW too small

In addition, IERR = −1 indicates that the maximum number of Lanczos steps in LANSEL has been reached.

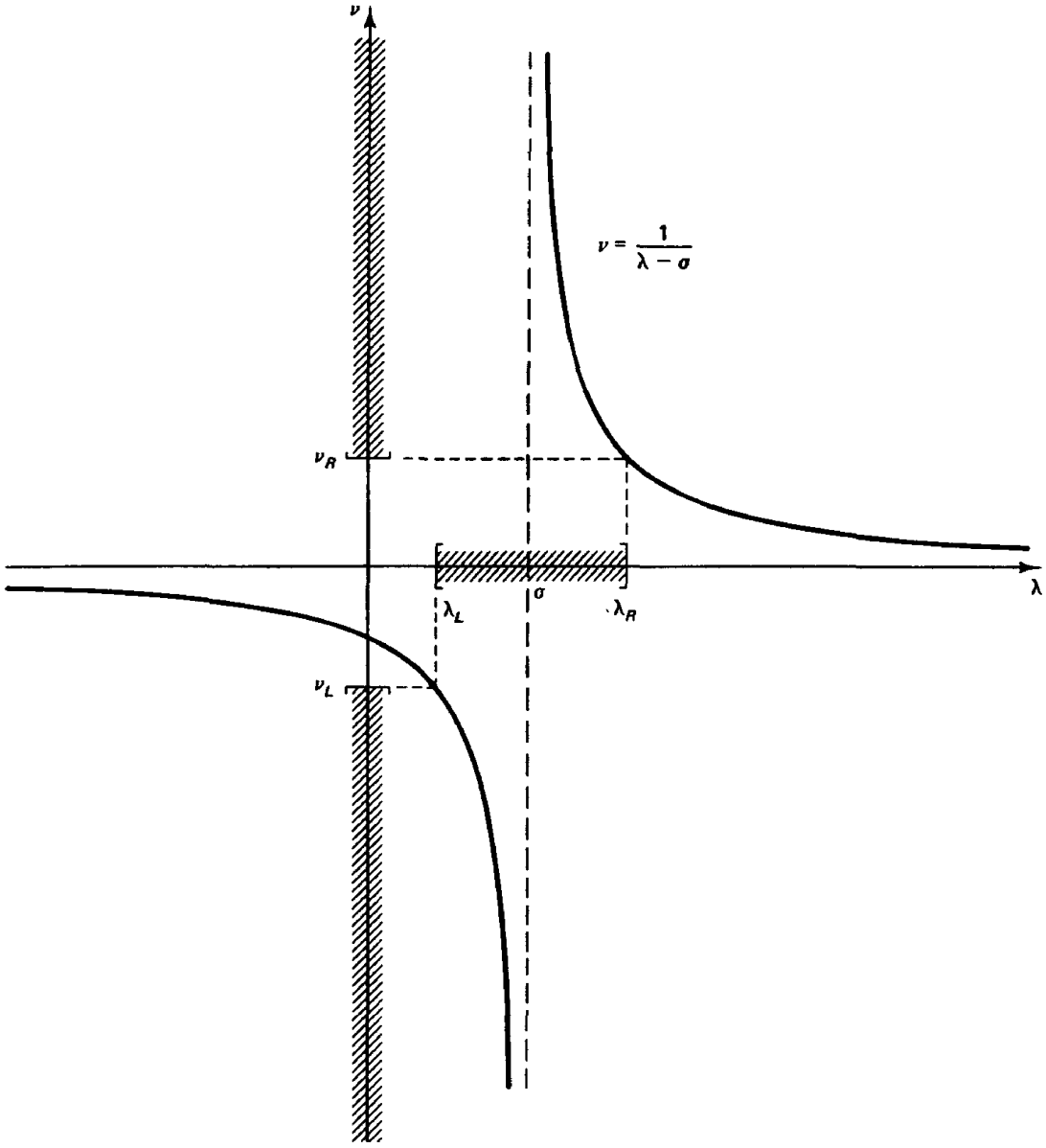


Figure 10.6.2 Spectral transformation of the interval containing the wanted eigenvalues; σ denotes the shift.

Remarks

1. LANSEL has no knowledge of the shift. The eigenvalues that it computes are those of the transformed problem (10.6.6). The eigenvalues of the original problem may be obtained by applying the inverse transformation $\lambda = \sigma + 1/\nu$. This is the responsibility of the user. The λ's closest to σ *usually* converge first. In particular, for a system with positive-definite $\boldsymbol{K}$ and $\boldsymbol{M}$, and $\sigma = 0$, the λ's *usually* converge in ascending order starting with the smallest.

2. LANSEL may return only a single copy of multiple eigenvalues. To obtain additional copies, restart LANSEL with a new starting vector orthogonal to the computed eigenvectors. (The number of missed eigenvalues may be determined by the technique of spectrum slicing; see Sec. 10.5.1.).

3. Certain rare execution errors, beyond the control of the Lanczos algorithm

may occur in subsidiary routines such as GIVENS. These are indicated by a returned value of IERR greater than 127. The only recourse is to rerun LANSEL with a different shift and/or starting vector.

```
C
      SUBROUTINE LANDRV(N,LANMAX,MAXPRS,ENDL,ENDR,NW,W,IW,EIG,Y,IERR,
     1                  NEIG)
C
CCCCCC            ******************************************
CCCCCC            *                                        *
CCCCCC            *             L  A  N  S  E  L           *
CCCCCC            *                                        *
CCCCCC            *          LANCZOS ALORITHM WITH         *
CCCCCC            *       SELECTIVE ORTHOGONALIZATION      *
CCCCCC            *                                        *
CCCCCC            *           B. N O U R - O M I D         *
CCCCCC            *                                        *
CCCCCC            ******************************************
CC.... INPUTS
CCCCCC      N        DIMENSION OF THE EIGENPROBLEM
CCCCCC      LANMAX   UPPER LIMIT TO THE NUMBER OF LANCZOS STEPS
CCCCCC      MAXPRS   UPPER LIMIT TO THE NUMBER OF WANTED EIGENPAIRS
CCCCCC      ENDL     LEFT END OF THE INTERVAL CONTAINING THE WANTED EIGENVALUES
CCCCCC      ENDR     RIGHT END OF THE INTERVAL CONTAINING THE WANTED EIGENVALUES
CCCCCC      NW       LENGTH OF THE WORK ARRAY W
CCCCCC      W        WORK ARRAY OF LENGTH NW
CCCCCC      IW       WORK ARRAY OF LENGTH MAXPRS
CCCCCC
CC.... OUTPUTS
CCCCCC
CCCCCC      EIG      ARRAY OF LENGTH MAXPRS TO HOLD THE CONVERGED RITZ VALUES
CCCCCC      Y        ARRAY OF LENGTH MAXPRS*N TO HOLD THE CONVERGED RITZ VECTORS
CCCCCC      IERR     ERROR FLAG
CCCCCC      NEIG     TOTAL NUMBER OF CONVERGED EIGENPAIRS
CCCCCC
CC.... SUBROUTINES: DDOT,GETEPS,LANSEL,OPM,RAN,STPONE
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION NQ(5),Y(1),EIG(1),W(1),IW(1)
      INTEGER*2 IRANDOM1,IRANDOM2
C
CC.... COMMON IDATA
CCCCCC
CCCCCC      EIGL     INNER MOST EIGENVALUE CONVERGED FROM LEFT END OF
CCCCCC               TRANSFORMED SPECTRUM
CCCCCC      EIGR     INNER MOST EIGENVALUE CONVERGED FROM RIGHT END OF
CCCCCC               TRANSFORMED SPECTRUM
CCCCCC      NEIG1    TOTAL NUMBER OF CONVEGED EIGENVALUES
C
      COMMON /IDATA/ EIGL, EIGR, NEIG1
C
CC.... COMMON RDATA
CCCCCC
CCCCCC      RNM      NORM OF THE RESIDUAL VECTOR IN R(NQ(1))
CCCCCC      RNM2     SQUARE OF RNM
CCCCCC      SPREAD   WIDTH OF THE INTERVAL CONTAINING THE UNCONVERGED
CCCCCC               EIGENVALUES
CCCCCC      TOL      TOLERANCE FOR CONVERGENCE OF THE EIGENVALUES
CCCCCC      EPS      COMPUTER PRECISION
CCCCCC      EPS1     EPS*SQRT(N)
CCCCCC      REPS     SQUARE ROOT OF EPS
C
      COMMON /RDATA/ RNM, RNM2, SPREAD, TOL, EPS, EPS1, REPS
C
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
      DATA ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO, FOUR, TEN,
     1     ONE28, TWO56, FIVE12, ORTFAC, OVRFLW /
     2     0.0D0, 0.1D0, 0.125D0, 0.25D0, 0.5D0, 1.0D0, 2.0D0, 4.0D0,
     3     10.0D0, 128.0D0, 256.0D0, 512.0D0, 4.0D0, 1.7014D+38/
C
CC.... CHECK INPUT DATA
C
      IERR = 0
      MT = 6*N + 2*MAXPRS + 10*LANMAX
      IF ( N .LE. 0 )              IERR = IERR + 1
      IF ( LANMAX .LE. 0 )         IERR = IERR + 2
      IF ( ENDR .LE. ENDL )        IERR = IERR + 4
      IF ( MAXPRS .LE. 0 )         IERR = IERR + 8
      IF ( MAXPRS .GT. LANMAX )    IERR = IERR + 16
```

```
      IF ( LANMAX .GT. N )        IERR = IERR + 32
      IF ( MT .GT. NW )           IERR = IERR + 64
      IF ( IERR .GT. 0 ) RETURN

      COMPUTE THE MACHINE PRECISION

      EPS  = GETEPS(IBETA,IT,IRND)

      REPS = SQRT(EPS)
      EPS1 = EPS*SQRT(FLOAT(N))

      SET POINTERS AND INITIALIZE

      M1 = 1       + 6*N
      M2 = MAXPRS + M1
      M3 = MAXPRS + M2
      M4 = LANMAX + M3
      M5 = LANMAX + M4
      M6 = LANMAX + M5
      M7 = LANMAX + M6
      M8 = LANMAX + M7
      M9 = LANMAX + M8
      M10 = LANMAX + M9
      M11 = LANMAX + M10

      IF MORE STORAGE IS TO BE ALLOCATED, MUST CHANGE "MT" ABOVE

      NS = NW - MT + 2*LANMAX
      NQ(1) = N + 1
      DO 20 I = 2,5
         NQ(I) = NQ(I-1) + N
      CONTINUE

      CHECK FOR STARTING VECTOR

      RNM2 = ZERO
      DO 30 I = 1,N
         RNM2 = RNM2 + ABS(W(I))
      CONTINUE

      IF (RNM2 .EQ. ZERO) THEN

         GET RANDOM STARTING VECTOR

         IRAND = N + LANMAX + MAXPRS + NW
       IRANDOM1=IRAND
         DO 40 I = 1,N
          IRANDOM2=I
            W(I) = RAN(IRANDOM1,IRANDOM2)
         CONTINUE
      END IF

      CALL OPM(W,W(NQ(3)),N)
      RNM2 = DDOT(N,W,1,W(NQ(3)),1)
      CALL STPONE(N,W(M3),W(M4),W(M5),W(M6),W,NQ)
      CALL LANSEL(N,LANMAX,MAXPRS,NS,ENDL,ENDR,W,W(M3),W(M4),W(M5),
     1            W(M6),EIG,W(M1),W(M2),W(M10),W(M11),W(M7),W(M8),
     2            IW,Y,W(M9),NQ,IERR)
      IF (IERR.GT.0) THEN
         RETURN
      ENDIF
      NEIG = NEIG1

      RETURN
      END
```

Function GETEPS

GETEPS determines the precision of the computer being used. It evaluates the unit roundoff error (i.e., the smallest number, ϵ, in the computer such that $1 + \epsilon > 1$).

```
      FUNCTION GETEPS(IBETA, IT, IRND)

      IMPLICIT DOUBLE PRECISION (A-H,O-Z)

      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW

      DETERMINE IBETA, BETA
```

```
      A = ONE
10    A = A + A
      IF (((A + ONE) - A) - ONE .EQ. ZERO) GO TO 10
      B = ONE
20    B = B + B
      IF ((A + B) - A .EQ. ZERO) GO TO 20
      IBETA = INT(SNGL((A + B) - A))
      BETA = FLOAT(IBETA)
C
C.... DETERMINE IT, IRND
C
      IT = 0
      B = ONE
30    IT = IT + 1
      B = B * BETA
      IF (((B + ONE) - B) - ONE .EQ. ZERO) GO TO 30
      IRND = 0
      BETAM1 = BETA - ONE
      IF ((A + BETAM1) - A .NE. ZERO) IRND = 1
C
C.... DETERMINE EPS
C
      BETAIN = ONE / BETA
      A = ONE
      DO 40 I = 1, IT + 3
         A = A * BETAIN
40    CONTINUE
C
50    IF ((ONE + A) - ONE .NE. ZERO) GO TO 60
      A = A * BETA
      GO TO 50
60    EPS = A
      IF ((IBETA .EQ. 2) .OR. (IRND .EQ. 0)) GO TO 70
      A = (A*(ONE + A)) / (ONE + ONE)
      IF ((ONE + A) - ONE .NE. ZERO) EPS = A
70    GETEPS = EPS
C
      RETURN
      END
```

Subroutine STPONE

This routine performs the initialization of ALF, BET, ALPH and BET2 as well as the first step of the Lanczos algorithm. See Sec. 10.6.5 for notation. Since $q_{j-1} = 0$ at the first step the algorithm reduces to an orthogonalization of $K_\sigma^{-1}Mq_1$ against q_1. q_1 is obtained by normalizing r and is stored in the array R starting at location NQ(1). Mr, stored in starting location NQ(3) of R, is then normalized to get Mq_1 which is put in R starting from NQ(4). NQ(2) and NQ(5) are the location of q_{j-1} and a temporary vector in R, respectively.

```
C
      SUBROUTINE STPONE(N,ALF,BET,ALPH,BET2,R,NQ)
C
C.... THIS ROUTINE PERFORMS THE FIRST STEP OF THE LANCZOS ALGORITHM.
C     IT PERFORMS A STEP OF LOCAL REORTHOGONALIZATION IF NEEDED.
C
C.... INPUT/OUTPUT
C
C     N       DIMENSION OF THE EIGENPROBLEM
C     ALF     THE NEW DIAGONAL OF T
C     BET     THE NEW OFF-DIAGONAL OF T
C     ALPH    THE NEW DIAGONAL OF THE DEFLATED T
C     BET2    THE NEW OFF-DIAGONAL SQUARED OF THE DEFLATED T
C     R       AN ARRAY CONTAINING [R(J),Q(J),Q(J-1),P(J),MR(J)]
C     NQ(5)   LOCATION POINTERS FOR THE ARRAY R
C
C.... SUBROUTINES: DAXPY,DCOPY,DDOT,DSCAL,OPK,OPM
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION NQ(1),R(1)
C
      COMMON /RDATA/ RNM,RNM2,SPREAD,TOL,EPS,EPS1,REPS
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
C.... MODIFY R TO SATISFY THE CORRECT CONDITIONS FOR SINGULAR M
C
```

```
T = ONE/SQRT(RNM2)
CALL DCOPY(N,R(NQ(3)),1,R(NQ(4)),1)
CALL DSCAL(N,T,R(NQ(4)),1)

CALL OPK(R(NQ(4)),R,N)
CALL OPM(R,R(NQ(3)),N)
RNM2 = DDOT(N,R,1,R(NQ(3)),1)
RNM  = SQRT(RNM2)

BET2(1) OUGHT TO BE ZERO.
BET(1) STORES THE M-NORM OF THE STARTING VECTOR.

BET2 = ZERO
BET = RNM
T = ONE/RNM
CALL DCOPY(N,R,1,R(NQ(1)),1)
CALL DSCAL(N,T,R(NQ(1)),1)
CALL DCOPY(N,R(NQ(3)),1,R(NQ(4)),1)
CALL DSCAL(N,T,R(NQ(4)),1)
CALL OPK(R(NQ(4)),R,N)

ALF = ZERO
DALF = DDOT(N,R,1,R(NQ(4)),1)
DO 10 I=1,2
   CALL DAXPY(N,-DALF,R(NQ(1)),1,R,1)
   ALF = ALF +DALF
   ALPH = ALF
   CALL OPM(R,R(NQ(3)),N)
   RNM2 = DDOT(N,R,1,R(NQ(3)),1)
   IF (I .EQ. 2) RETURN

   DALF = DDOT(N,R(NQ(1)),1,R(NQ(3)),1)
   DBET = DDOT(N,R(NQ(2)),1,R(NQ(3)),1)
   CALL DAXPY(N,-DBET,R(NQ(2)),1,R,1)
CONTINUE

RETURN
END
```

Subroutine LANSEL

LANSEL, LANczos algorithm with SELective orthogonalization, performs the main steps of the procedure described in the previous sections. The flowchart of Fig. 10.6.3 gives a global structure of LANSEL. After some initializations the algorithm checks if any orthogonalization is necessary by calling PURGE. Then it calls LANSIM to perform a step of simple Lanczos. The orthogonality estimates and the eigenvalues of the tridiagonal matrix are updated by calling ORTBND and ANALZT, respectively. When enough eigenvalues have been computed loop 10 is terminated and finally the wanted eigenvectors are computed using RITVEC.

LANSEL terminates the iteration if any of the following conditions are satisfied:

a. The norm of r_j becomes small (indicating an invariant subspace).
b. The number of Lanczos steps reaches LANMAX, the maximum allowed. In this case IERR is set to -1.
c. The desired eigenvalues are computed (see LOGICAL FUNCTION ENOUGH).

If the interval [ENDL, ENDR] contains no eigenvalue, then LANSEL will continue until an eigenvalue has converged outside each end of this interval. If the first MAXPRS eigenvalues are wanted, then set ENDL = $-\infty$, and ENDR = $+\infty$ (i.e., set to very large negative and positive numbers, respectively).

Structure of R: The first vector starting at location 1 is r_j when entering the subroutine. The remaining five vectors are stored at locations NQ(1) through NQ(5).

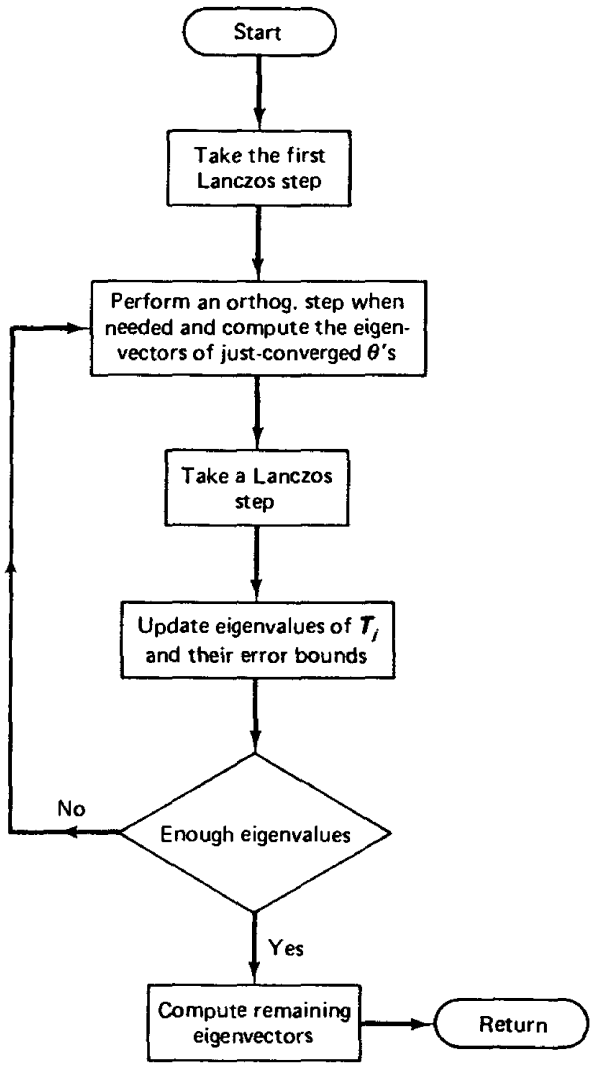


Figure 10.6.3 A global picture of the Lanczos method with selective orthogonalization indicating the way the individual modules are used.

This eliminates the need for moving the content of one array to another for swapping. Only the pointers of the arrays are changed. The vectors q_j, q_{j-1}, Mr_j and Mq_j are held in R starting from locations NQ(1) through NQ(4), respectively. The space in R starting from NQ(5) is used for a working vector in other parts of the program.

```
C
      SUBROUTINE LANSEL(N,LANMAX,MAXPRS,NS,ENDL,ENDR,R,ALF,BET,ALPH,
     1                  BET2,EIG,TAU,OLDTAU,RHO,WORK,ETA,OLDETA,INFO,
     2                  Y,S,NQ,IERR)
C
C.... INPUTS
C
C     N        DIMENSION OF THE EIGENPROBLEM
C     LANMAX   UPPER LIMIT TO THE NUMBER OF LANCZOS STEPS
C     MAXPRS   UPPER LIMIT TO THE NUMBER OF WANTED EIGENPAIRS
C     NS       LENGTH OF THE ARRAY S
C     ENDL     LEFT END OF THE INTERVAL CONTAINING THE WANTED EIGENVALUES
C     ENDR     RIGHT END OF THE INTERVAL CONTAINING THE WANTED EIGENVALUES
C
C.... WORK SPACE
C
C     R        HOLDS 6 VECTORS OF LENGTH N. SEE THE TEXT FOR DETAILS.
C     NQ(5)    CONTAINS THE POINTERS TO THE BEGINING OF EACH VECTOR IN R.
C     ALF      ARRAY OF LENGTH LANMAX TO HOLD DIAGONAL OF THE TRIDIAGONAL T
C     BET      ARRAY OF LENGTH LANMAX TO HOLD OFF-DIAGONAL OF T
C     ALPH     DIAGONAL OF THE DEFLATED TRIDIAGONAL
C     BET2     SQUARE OF THE OFF-DIAGONALS OF THE DEFLATED TRIDIAGONAL
C     TAU      ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J
```

```
C     OLDTAU  ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J-1
C     RHO     WORKING ARRAY USED IN DEFLAT()
C     ETA     ORTHOGONALITY ESTIMATE OF LANCZOS VECTORS AT STEP J
C     OLDETA  ORTHOGONALITY ESTIMATE OF LANCZOS VECTORS AT STEP J-1
C     INFO    INFORMATION ARRAY ABOUT EIGENVECTORS OF T
C     S       ARRAY FOR COMPUTING THE EIGENVECTORS OF THE TRIDIAGONAL
C     WORK    WORKING ARRAY TO HOLD SQUARES OF ARRAY BETA
C
C.... OUTPUTS
C
C     EIG     ARRAY OF LENGTH MAXPRS TO HOLD THE CONVERGED RITZ VALUES
C     Y       ARRAY OF LENGTH MAXPRS*N TO HOLD THE CONVERGED RITZ VECTORS
C     NEIG    NUMBER OF COMPUTED EIGENPAIRS
C     IERR    ERROR FLAG
C
C.... SUBROUTINES: ANALZT,ENOUGH,LANSIM,ORTBND,PURGE,RITVEC
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION R(1),Y(N,1),EIG(1),TAU(1),OLDTAU(1),ALF(1),BET(1)
      DIMENSION S(1),NQ(1),ALPH(1),BET2(1),ETA(1),OLDETA(1),INFO(1)
      DIMENSION RHO(1),WORK(1)
      LOGICAL LASTEP,ENOUGH
C
      COMMON /RDATA/ RNM,RNM2,SPREAD,TOL,EPS,EPS1,REPS
      COMMON /IDATA/ EIGL,EIGR,NEIG
C
      JJ      = 1
      ETA(1)  = EPS1
      EIGL    = ENDL - (ENDR - ENDL)
      EIGR    = ENDR + (ENDR - ENDL)
      NEIG    = 0
C
C.... LANCZOS LOOP
C
      DO 10 J = 2,LANMAX
         NBUF = NS/J
         RNM  = SQRT(RNM2)
C
C....    RESTORE THE ORTHOGONALITY STATE WHEN NEEDED
C
         CALL PURGE(R,R(NQ(1)),R(NQ(3)),R(NQ(4)),R(NQ(5)),Y,ALF,BET,S,
     1              EIG,ETA,OLDETA,TAU,OLDTAU,WORK,INFO,N,J-1,NBUF,IERR)
         IF (IERR .GT. 0) RETURN
C
C....    UPDATE THE RITZ VALUES
C
         CALL ANALZT(JJ,ALPH,BET2,EIG,TAU,OLDTAU,RHO,INFO)
C
         IF ( ENOUGH(ENDL,ENDR,MAXPRS) ) GO TO 30
C
         IF (RNM .LT. REPS*SPREAD) GO TO 20
         JJ   = JJ + 1
C
C....    TAKE A LANCZOS STEP
C
         CALL LANSIM(R,ALF(J),BET(J),ALPH(JJ),BET2(JJ),RNM,RNM2,NQ,
     1               N,J)
C
C....    UPDATE THE ORTHOGONALITY BOUNDS
C
         CALL ORTBND(ALF,BET,J,EPS1,ETA,OLDETA,TAU,OLDTAU,EIG,INFO,
     1               RNM,NEIG,N)
C
10    CONTINUE
20    J = J - 1
C
C.... COMPUTE THE REMAINING RITZ VECTORS
C
30    DO 50 I=1,NEIG
         M = 1
         DO 40 K = 1,NEIG
            IF ( INFO(K) .GT. 0 ) INFO(K) = -INFO(K)
            M =  MIN(ABS(INFO(K)),M)
40       CONTINUE
C
         IF ( M .EQ. 0 ) THEN
            CALL RITVEC(R,R(NQ(1)),R(NQ(3)),R(NQ(4)),R(NQ(5)),Y,ALF,
     1           BET,EIG,S,INFO,N,J,NEIG,NBUF,.TRUE.,WORK,IERR)
            IF (IERR .GT. 0) THEN
               RETURN
            ENDIF
         ELSE
            GO TO 60
         END IF
C
50    CONTINUE
```

```
C
60    CONTINUE
      IF( J .EQ. LANMAX ) THEN
      IERR = -1
      END IF
C
      RETURN
      END
```

Logical Function ENOUGH

ENOUGH determines if all the desired eigenvalues have converged. This is established if one of the following conditions is satisfied:

a. The number of computed eigenvalues exceeds the maximum number requested, MAXPRS.

b. An eigenvalue is found outside the interval [ENDL, ENDR] at each end.

```
C
      LOGICAL FUNCTION ENOUGH(ENDL,ENDR,MAXPRS)
C
C.... EXAMINE IF ENOUGH EIGENVALUES HAVE CONVERGED
C
C.... INPUT
C
C     ENDL    LEFT END OF THE INTERVAL CONTAINING THE WANTED EIGENVALUES
C     ENDR    RIGHT END OF THE INTERVAL CONTAINING THE WANTED EIGENVALUES
C     MAXPRS  UPPER LIMIT TO THE NUMBER OF WANTED EIGENPAIRS
C
      IMPLICIT DOUBLE PRECISION(A-H,O-Z)
      PARAMETER (NMAX = 128)
C
      COMMON/ATDATA/THET(NMAX),BJ(NMAX),WINDOW,NBD(2),NDST
      COMMON/IDATA/EIGL,EIGR,NEIG
C
      ENOUGH = .TRUE.
      IF ( NEIG .GT. MAXPRS ) RETURN
C
      ENOUGH = ( THET(1) .GT. ENDL .AND. THET(NDST) .LT. ENDR ) .AND.
     1          (( EIGL .GT. ENDL .AND. EIGL .LT. ENDR ) .OR.
     2           ( EIGR .GT. ENDL .AND. EIGR .LT. ENDR ))
C
      RETURN
      END
```

Subroutine LANSIM

This routine performs a single step of the simple Lanczos process. The flowchart in Fig. 10.6.4 gives a global view of the algorithm. All the vectors required in this routine are stored in array R.

```
C
      SUBROUTINE LANSIM(R,ALF,BET,ALPH,BET2,RNM,RNM2,NQ,N,J)
C
C.... THIS ROUTINE PERFORMS A SINGLE STEP OF THE LANCZOS ALGORITHM,
C     FOLLOWED BY A STEP OF LOCAL REORTHOGONALIZATION IF NEEDED.
C
C.... INPUT/OUTPUT
C
C     R       AN ARRAY CONTAINING [R(J),Q(J),Q(J-1),P(J),MR(J)]
C     ALF     THE NEW DIAGONAL OF T
C     BET     THE NEW OFF-DIAGONAL OF T
C     ALPH    THE NEW DIAGONAL OF THE DEFLATED T
C     BET2    THE NEW OFF-DIAGONAL SQUARED OF THE DEFLATED T
C     RNM     NORM OF R(J)
C     RNM2    RNM**2
C     NQ(5)   LOCATION POINTERS FOR THE ARRAY R
C     N       DIMENSION OF THE EIGENPROBLEM
C     J       CURRENT LANCZOS STEP
C
C.... SUBROUTINES: DAXPY,DCOPY,DDOT,DSCAL,OPK,OPM,STORE
C
```

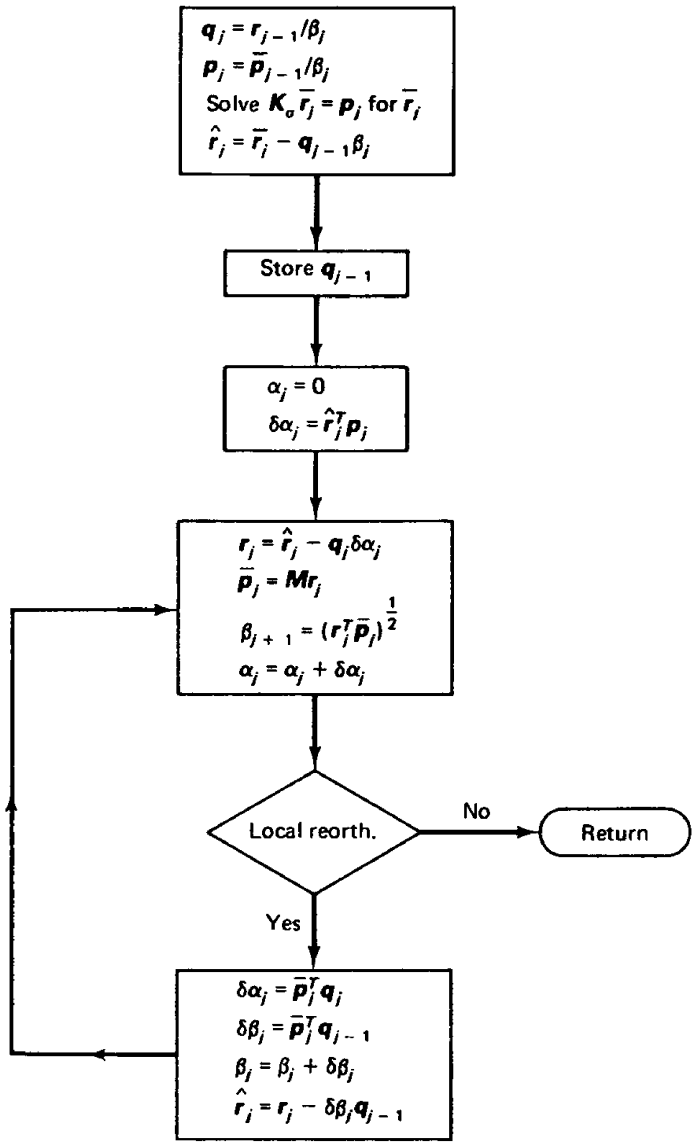


Figure 10.6.4 Flowchart description of the jth step of the simple Lanczos algorithm.

```
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION NQ(1),R(1)
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C..   SWAP Q(J) AND Q(J-1)
      NTMP = NQ(2)
      NQ(2) = NQ(1)
      NQ(1) = NTMP
C..   Q = R/BETA
      T = ONE/RNM
      CALL DCOPY(N,R,1,R(NQ(1)),1)
      CALL DSCAL(N,T,R(NQ(1)),1)
C..   P = PBAR/BETA
      CALL DCOPY(N,R(NQ(3)),1,R(NQ(4)),1)
      CALL DSCAL(N,T,R(NQ(4)),1)
C..   R = ( K INVERSE ) * Q
      CALL OPK(R(NQ(4)),R,N)
C..   R = R - Q(J-1)*BETA
```

```
C
      T = -RNM
      CALL DAXPY(N,T,R(NQ(2)),1,R,1)
C
C.... STORE Q(J-1)
C
      CALL STORE(R(NQ(2)),N,J-1,1)
C
C.... START LOCAL REORTHOGONALIZATION
C
      BET = RNM
      BET2 = RNM2
      ALF = ZERO
C
C.... ALF = ( R TRANSPOSE ) * P
C
      DALF = DDOT(N,R,1,R(NQ(4)),1)
      DO 10 I=1,2
         CALL DAXPY(N,-DALF,R(NQ(1)),1,R,1)
         ALF = ALF + DALF
         CALL OPM(R,R(NQ(3)),N)
         RNM2 = DDOT(N,R,1,R(NQ(3)),1)
         ALPH = ALF
C
         IF (RNM2*ORTFAC .GT. (ALF**2 + BET2) .OR. I .EQ. 2) RETURN
C
C....    REPEAT LOCAL REORTHOGONALIZATION WHEN WARRANTED
C
         DALF = DDOT(N,R(NQ(1)),1,R(NQ(3)),1)
         DBET = DDOT(N,R(NQ(2)),1,R(NQ(3)),1)
         CALL DAXPY(N,-DBET,R(NQ(2)),1,R,1)
         BET = BET + DBET
         BET2 = BET**2
10    CONTINUE
C
      END
```

Subroutine ORTBND

The subroutine ORTBND updates the orthogonality bounds, h_j, of (10.6.36), and the τ recurrence of (10.6.40). These quantities are later examined in subroutine PURGE for possible loss of semiorthogonality. LANSEL is the only routine that calls ORTBND.

```
C
      SUBROUTINE ORTBND (ALF,BET,J,EPS1,ETA,OLDETA,TAU,OLDTAU,EIG,INFO,
     1                   RNM,NEIG,N)
C
C.... UPDATE THE ETA AND TAU RECURRENCES.
C
C.... INPUTS
C
C     ALF(J)         DIAGONAL OF THE TRIDIAGONAL T
C     BET(J)         OFF-DIAGONAL OF T
C     J              DIMENSION OF T
C     EPS1           ROUNDOFF ESTIMATE FOR DOT PRODUCT OF TWO UNIT VECTORS
C     ETA(J)         ORTHOGONALITY ESTIMATE OF LANCZOS VECTORS AT STEP J
C     OLDETA(J)      ORTHOGONALITY ESTIMATE OF LANCZOS VECTORS AT STEP J-1
C     TAU(NEIG)      ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J
C     OLDTAU(NEIG)   ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J-1
C     EIG(NEIG)      ARRAY OF CONVERGED EIGENVALUES
C     INFO(NEIG)     INFORMATION ARRAY ABOUT EIGENVECTORS OF T
C     RNM            NORM OF THE NEXT RESIDUAL VECTOR
C     NEIG           NUMBER OF CONVERGED EIGENVALUES
C     N              DIMENSION OF THE EIGENPROBLEM
C
C.... OUTPUTS
C
C     ETA(J)         ORTHOGONALITY ESTIMATE OF LANCZOS VECTORS AT STEP J+1
C     OLDETA(J)      ORTHOGONALITY ESTIMATE OF LANCZOS VECTORS AT STEP J
C     TAU(NEIG)      ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J+1
C     OLDTAU(NEIG)   ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION ALF(1),BET(1),ETA(1),OLDETA(1),TAU(1),OLDTAU(1)
      DIMENSION EIG(1),INFO(1)
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
```

```
      IF ( J .GT. 1 ) THEN
         OLDETA(1) = ( BET(2)*ETA(2) + (ALF(1) - ALF(J))*ETA(1)
     |                 - BET(J)*OLDETA(1)) / RNM
         J1 = J - 1
         IF ( J .GT. 2 )  THEN
            DO 100 K=2,J1
               OLDETA(K) = (BET(K+1)*ETA(K+1) + (ALF(K) - ALF(J))*ETA(K)
     |                     + BET(K)*ETA(K-1) - BET(J)*OLDETA(K)) / RNM
  100       CONTINUE
         END IF
         DO 200 K=1,J1
               T = OLDETA(K)
               OLDETA(K) = ETA(K)
               ETA(K) = T
  200    CONTINUE
      END IF
      ETA(J) = EPS1*MAX(BET(2)/RNM,ONE)

      UPDATE THE TAU RECURRENCE.

      DO 300 I=1,NEIG
         IF ( INFO(I) .NE. 0 ) THEN
            T = TAU(I)
            TAU(I) = (EIG(I) - ALF(J))*TAU(I) - BET(J)*OLDTAU(I)
            OLDTAU(I) = T

         END IF
  300 CONTINUE

      RETURN
      END
```

Subroutine PURGE

PURGE first examines the array ETA, which holds the vector $\boldsymbol{h}_{j+1}$. See Sec. 10.6.8. If the element of ETA with largest absolute value is less then REPS, $\sqrt{\epsilon}$, indicating no loss of semiorthogonality, then PURGE does nothing and returns. Otherwise, the elements of TAU are examined to determine if loss of orthogonality might be due to the return of a previously banished Ritz vector. If TAU(I) is greater than REPS in absolute value, it indicates loss of semiorthogonality against the Ritz vector with index I. The corresponding eigenvector of $\boldsymbol{T}_j$ is computed using GIVENS. ETA and OLDETA are orthogonalized against the computed eigenvector of $\boldsymbol{T}_j$. See Fig. 10.6.5.

The elements of ETA are examined for a second time. If ETA still holds an element with absolute value greater than REPS, then loss of orthogonality is also due to the convergence of a Ritz value. Then RITVEC is called and the contents of TAU, OLDTAU, ETA, and OLDETA are all set to EPS1 $= \sqrt{n\epsilon}$. Otherwise, $\boldsymbol{q}_j$ and $\boldsymbol{r}_j$, held in Q and R, are orthogonalized against those Ritz vectors in columns of Y with TAU value greater than REPS in absolute value. These elements of TAU and OLDTAU are then set to EPS1.

All orthogonalizations of the Lanczos vectors are performed with respect to the inertial inner product, and therefore we require two vectors, QA and RA, that hold $\boldsymbol{Mq}_j$ and $\boldsymbol{Mr}_j$, respectively. At the end of PURGE, $\boldsymbol{Mq}_j$ and $\boldsymbol{Mr}_j$ are recomputed if $\boldsymbol{q}_j$ and $\boldsymbol{r}_j$ are modified.

```
      SUBROUTINE PURGE(R,Q,RA,QA,T,Y,ALF,BET,S,EIG,ETA,OLDETA,TAU,
     |                 OLDTAU,WORK,INFO,N,J,NBUF,IERR)

      THIS ROUTINE EXAMINES ETA, OLDETA, TAU AND OLDTAU TO DECIDE
      WHICH FORM OF REORTHOGONALIZATION IF ANY SHOULD BE PERFORMED.

      INPUT/OUTPUT

      R       THE RESIDUAL VECTOR TO BECOME THE NEXT LANCZOS VECTOR
      Q       THE CURRENT LANCZOS VECTOR
      RA      THE PRODUCT OF THE MASS MATRIX AND R
      QA      THE PRODUCT OF THE MASS MATRIX AND Q
```

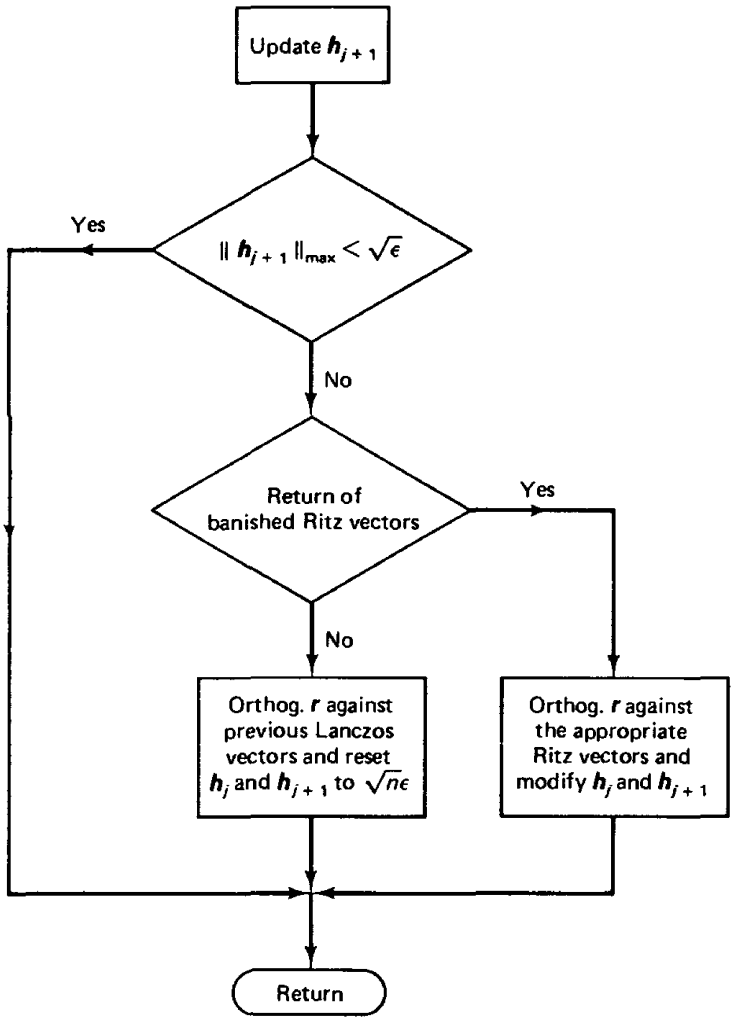


Figure 10.6.5 A flowchart for the selective orthogonalization method.

```
C     T       A TEMPORARY VECTOR TO HOLD THE PREVIOUS LANCZOS VECTORS
C     Y       CONTAINS THE COMPUTED RITZ VECTORS
C     ALF     THE NEW DIAGONAL OF T
C     BET     THE NEW OFF-DIAGONAL OF T
C     S       VECTOR FOR COMPUTING EIGENVECTORS OF T(J)
C     EIG     HOLDS THE CONVERGED RITZ VALUES
C     ETA     STATE OF ORTHOGONALITY BETWEEN R AND PREVIOUS LANCZOS VECTORS
C     OLDETA  STATE OF ORTHOGONALITY BETWEEN Q AND PREVIOUS LANCZOS VECTORS
C     TAU     STATE OF ORTHOGONALITY BETWEEN R AND COMPUTED RITZ VECTORS
C     OLDTAU  STATE OF ORTHOGONALITY BETWEEN Q AND COMPUTED RITZ VECTORS
C     WORK    WORKING ARRAY EXPLAINED IN LANSEL
C     INFO    INFORMATION ABOUT THE EIGENVECTORS OF T(J)
C     N       DIMENSION OF THE EIGENPROBLEM
C     J       CURRENT LANCZOS STEP
C     NEIG    NUMBER OF RITZ VALUES
C     NBUF    NUMBER OF VECTORS IN S
C
C.... SUBROUTINES: DAXPY,DZERO,DDOT,GIVENS,IDAMAX,OPM,RITVEC,SUBTJ
C
      IMPLICIT DOUBLE PRECISION(A-H,O-Z)
      DIMENSION R(1),Q(1),RA(1),QA(1),T(1),Y(N,1),ALF(1),BET(1),S(J,1)
      DIMENSION EIG(1),ETA(1),OLDETA(1),TAU(1),OLDTAU(1),INFO(1),WORK(1)
      LOGICAL ORTHO
C
      COMMON/IDATA/EIGL,EIGR,NEIG
      COMMON/RDATA/RNM,RNM2,SPREAD,DUMMY,EPS,EPS1,REPS
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
      ORTHO  = .FALSE.
      TOL    = REPS*SPREAD
      REPSOJ = SQRT(EPS/FLOAT(J))
      K      = IDAMAX(J-2,ETA,1)
      IF (ABS(ETA(K)) .GT. REPSOJ) THEN
```

```
   DO 10 I=1,NEIG
      IF (INFO(I) .LT. 0 .AND. ABS(TAU(I)) .GT. REPSOJ) THEN
         CALL DZERO(J,S,1)
         CALL SUBTJ(ALF,BET,EIG,INFO,TOL,I,N,NEIG,J,L,K,M,
                    WORK,IERR)
         IF (IERR .GT. 0) RETURN
         CALL GIVENS(K-M+1,L-M+1,ALF(M),BET(M),EIG(I),EPS,
                     S,RESID,RAYCOR,IERR)
         IF (IERR .GT. 0) RETURN
         ZETA = -DDOT(J,ETA,1,S,1)
         CALL DAXPY(J,ZETA,S,1,ETA,1)
         ZETA = -DDOT(J,OLDETA,1,S,1)
         CALL DAXPY(J,ZETA,S,1,OLDETA,1)
      END IF
   CONTINUE

   K = IDAMAX(J-2,ETA,1)

   IF (ABS(ETA(K)) .GT. REPSOJ) THEN

      GRAM-SCHMID NEEDED

      ORTHO = .TRUE.
      CALL RITVEC(R,Q,RA,QA,T,Y,ALF,BET,EIG,S,INFO,N,J,NEIG,
                  NBUF,.FALSE.,WORK,IERR)
      IF (IERR.GT.0) RETURN
      DO 20 I=1,NEIG
         TAU(I) = EPS1
         OLDTAU(I) = EPS1
      CONTINUE
      DO 30 I=1,J-1
         ETA(I) = EPS1
         OLDETA(I) = EPS1
      CONTINUE
   ELSE

      REMOVE COMPONENTS OF A RITZ VECTOR

      ORTHO = .TRUE.
      DO 40 I = 1,NEIG
         IF (ABS(TAU(I)).GT.REPSOJ)THEN
            TAU(I) = EPS1
            OLDTAU(I) = EPS1
            ZETA = DDOT(N,RA,1,Y(1,I),1)
            CALL DAXPY(N,-ZETA,Y(1,I),1,R,1)
            ZETA = DDOT(N,QA,1,Y(1,I),1)
            CALL DAXPY(N,-ZETA,Y(1,I),1,Q,1)
         END IF
      CONTINUE
   END IF
END IF
IF (ORTHO) THEN
   CALL OPM(Q,QA,N)
   QNORM = SQRT(DDOT(N,Q,1,QA,1))
   CALL DSCAL(N,ONE/QNORM,Q,1)
   CALL DSCAL(N,ONE/QNORM,QA,1)
   BET(J) = BET(J) * QNORM
   ZETA   = DDOT(N,R,1,QA,1)
   ALF(J) = ALF(J) + ZETA
   CALL DAXPY(N,-ZETA,Q,1,R,1)
   CALL OPM(R,RA,N)
   RNM2 = DDOT(N,R,1,RA,1)
   RNM  = SQRT(RNM2)
END IF

RETURN
END
```

Subroutine RITVEC

The purpose of this routine is twofold:

1. Compute the Ritz vector corresponding to a converged Ritz value.
2. Perform a full reorthogonalization step.

The first step of the program is to compute some eigenvectors of the tridiagonal T_j. The eigenvectors of T_j are stored in an array S, and the number of computed vectors

depends on the available storage indicated by NBUF. GIVENS is used to obtain the wanted eigenvectors. The next step is to recall all the Lanczos vectors from secondary storage (by calling STORE) and compute the Ritz vectors using (10.6.23), accumulating the result in columns of Y. The old Lanczos vector that is brought in is stored in the temporary vector T. In the same loop, when EVONLY is false, the two current Lanczos vectors, q_j and r_j, held in Q and R, are orthogonalized against the previous Lanczos vectors held in T. In this way the number of I/O transfers is minimized. Arrays QA and RA that hold Mq_j and Mr_j, respectively, are also needed by RITVEC to perform the orthogonalization with respect to the inertial inner product.

```
C
      SUBROUTINE RITVEC(R,Q,RA,QA,T,Y,ALF,BET,EIG,S,INFO,N,J,NEIG,NBUF,
     1                  EVONLY,WORK,IERR)
C
C.... THIS ROUTINE COMPUTES SOME RITZ VECTORS AND PERFORMS A
C     REORTHOGONALIZATION OF THE LANCZOS VECTORS.
C
C.... INPUT/OUTPUT
C
C     R       THE RESIDUAL VECTOR TO BECOME THE NEXT LANCZOS VECTOR
C     Q       THE CURRENT LANCZOS VECTOR
C     RA      THE PRODUCT OF THE MASS MATRIX AND R
C     QA      THE PRODUCT OF THE MASS MATRIX AND Q
C     T       A TEMPORARY VECTOR TO HOLD THE PREVIOUS LANCZOS VECTORS
C     Y       CONTAINS THE COMPUTED RITZ VECTORS
C     ALF     THE NEW DIAGONAL OF T
C     BET     THE NEW OFF-DIAGONAL OF T
C     EIG     HOLDS THE CONVERGED RITZ VALUES
C     S       VECTOR FOR COMPUTING EIGENVECTORS OF T(J)
C     INFO    INFORMATION ABOUT THE EIGENVECTORS OF T(J)
C     N       DIMENSION OF THE EIGENPROBLEM
C     J       CURRENT LANCZOS STEP
C     NEIG    NUMBER OF RITZ VALUES
C     NBUF    NUMBER OF VECTORS IN S
C     EVONLY IF .TRUE. NO REORTHO. IS PERFORMED, COMPUTES ONLY RITZ VECTORS
C
C.... SUBROUTINES: DAXPY,DDOT,DZERO,GIVENS,IDAMAX,NUMLES,STORE
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION R(1),Q(1),RA(1),QA(1),T(1),Y(N,1),ALF(1),BET(1),S(J,1)
      DIMENSION EIG(1),INFO(1),WORK(1)
      LOGICAL EVONLY
C
      COMMON /RDATA/ RNM,RNM2,SPREAD,DUMMY,EPS,EPS1,REPS
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
C     EPS14 = EPS**(1/4)
C     TOL14 = EPS**(1/4)*SPREAD
C     TOL34 = EPS**(3/4)*SPREAD
C
      EPS14 = SQRT(REPS)
      TOL14 = EPS14*SPREAD
      TOL34 = REPS*TOL14
C
      IBUF = 0
      TOL = REPS*SPREAD
      RNEPS = RNM*EPS
C
C.... COMPUTE EIGENVECTORS OF T AND PUT IN THE BUFFER.
C
      DO 30 I=NEIG,1,-1
         IF (INFO(I) .GE. 0 .AND. IBUF .LT. NBUF) THEN
            IF (EVONLY .AND. INFO(I) .NE. 0) GO TO 30
            IBUF = IBUF + 1
            CALL DZERO(N,Y(1,I),1)
C
C....       EIG(I) ISOLATED
C
            M = 1
            L = J
            K = M
            CALL DZERO(J,S(1,IBUF),1)
            CALL GIVENS(K-M+1,L-M+1,ALF(M),BET(M),EIG(I),EPS1,
     1           S(M,IBUF),RESID,RAYCOR,IERR)
            IF (IERR .GT. 0) GO TO 900
            IF (RESID .LE. TOL14) GO TO 15
C
```

```
      CHECK THAT EIG(I) IS AN EIGENVALUE OF T

      DO 5 IDUM = M, L
         WORK(IDUM) = BET(IDUM)*BET(IDUM)
      CONTINUE
      ZETA = EIG(I)*(ONE - EPS14)
      NUL = NUMLES(ALF(M),WORK(M),ZETA,L-M+1,1,EPS)
      ZETA = EIG(I)*(ONE + EPS14)
      NUR = NUMLES(ALF(M),WORK(M),ZETA,L-M+1,1,EPS)

      EIG(I) IS NOT AN EIGENVALUE OF T, GOODBYE.

      IF (NUR .EQ. NUL) THEN
         IERR = IERR + 1024
         GO TO 900
      ENDIF

      EIG(I) IS O.K., NOW FIND A GOOD K

      DO 10 K=M+1,L
         CALL DZERO(J,S(1,IBUF),1)
         CALL GIVENS(K-M+1,L-M+1,ALF(M),BET(M),EIG(I),EPS1,
                     S(M,IBUF),RESID,RAYCOR,IERR)
         IF (IERR.GT.0) GO TO 900
         IF (RESID .LE. TOL14) GO TO 15
      CONTINUE

      NO SUITABLE K WAS FOUND, GOODBYE.

      IERR = IERR + 512
      GO TO 900

      NOW WE CAN REFINE THE EIGENVECTOR

      DO 20 LOOP = 1,10
         K = IDAMAX(L-M+1,S(M,IBUF),1)+M-1
         CALL DZERO(J,S(1,IBUF),1)
         CALL GIVENS(K-M+1,L-M+1,ALF(M),BET(M),EIG(I),EPS1,
                     S(M,IBUF),RESID,RAYCOR,IERR)
         IF (RESID.LE.TOL) GO TO 25
         EIG(I) = EIG(I) + RAYCOR
      CONTINUE

      RESID FAILED TO DIMINISH, GOODBYE.

      IF (RESID .GT. TOL) THEN
         IERR = IERR + 2048
         GO TO 900
      ENDIF

      INFO(I) = N*IDAMAX(J,S(1,IBUF),1)+J-1
   END IF
CONTINUE

COMPUTE THE RITZ VECTORS AND PERFORM G-S ORTHOGONALIZATION.

DO 50 I=1,J-1

   RETRIEVING THE LANCZOS VECTOR AND PUT IT IN T

   CALL STORE(T,N,I,2)
   KBUF = 0
   DO 40 K=NEIG,1,-1
      IF (INFO(K) .GE. 0 .AND. KBUF .LT. IBUF) THEN
         KBUF = KBUF + 1
         SI = S(I,KBUF)
         IF (ABS(SI) .GT. EPS) CALL DAXPY(N,SI,T,1,Y(1,K),1)
      END IF
   CONTINUE
   IF (.NOT.EVONLY) THEN
      ZETOLD = -DDOT(N,QA,1,T,1)
      IF ( ABS(ZETOLD) .GT. EPS ) CALL DAXPY(N,ZETOLD,T,1,Q,1)
      ZETA = -DDOT(N,RA,1,T,1)
      IF ( ABS(ZETA) .GT. RNEPS ) CALL DAXPY(N,ZETA,T,1,R,1)
   END IF
CONTINUE

ADD IN CONTRIBUTION OF QJ TO Y

KBUF = 0
DO 60 I=NEIG,1,-1
   IF (INFO(I) .GE. 0 .AND. KBUF .LT. IBUF) THEN
      KBUF = KBUF + 1
      IF (ABS(S(J,KBUF)) .LT. EPS1) INFO(I) = -INFO(I)
      CALL DAXPY(N,S(J,KBUF),Q,1,Y(1,I),1)
   END IF
```

```
60    CONTINUE
C
900   RETURN
      END
```

Subroutine GIVENS

This routine computes the eigenvector of a tridiagonal corresponding to an eigenvalue stored in THET. It first assumes a value for the bottom element of the eigenvector and solves for the rest using the Givens recurrence [28] going backward. This phase is terminated once the Kth element of S has been obtained. Then all the terms computed thus far are scaled to make S(K) unity. A similar procedure is performed starting from the top element and running the recurrence in the forward direction. Then the computed vector is normalized, and finally the residual and the Rayleigh correction to THETA are computed and returned in RES and COR, respectively. The algorithm scales S to avoid overflow whenever it becomes necessary.

```
C
      SUBROUTINE GIVENS(K,J,ALF,BET,THET,EPS,S,RES,COR,IERR)
C
C.... GIVENS RECURRENCE FOR COMPUTING EIGENVECTORS OF A TRIDIAGONAL T.
C
C.... INPUTS
C
C     K       INDEX OF THE RIGHT HAND SIDE E(K)
C     J       DIMENSION OF THE TRIDIAGONAL MATRIX
C     ALF(J)  DIAGONALS OF T
C     BET(J)  OFF-DIAGONALS OF T
C     THET    EIGENVALUE OF T
C     EPS     COMPUTER PRECISION
C
C.... OUTPUTS
C
C     S(J)    COMPUTED EIGENVECTOR
C     RES     NORM OF THE RESIDUAL
C     COR     RAYLEIGH CORRECTION FOR THET
C
C.... SUBROUTINES : DSCAL
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION ALF(J),BET(J),S(J)
C
C.... OVRFLW IS THE MACHINE OVERFLOW THRESHOLD
C
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
C.... BACKWARD RECURRENCE
C
      BIG = SQRT(OVRFLW/FLOAT(J+1))
      RES = ZERO
C
      S(J) = ONE
      SUM2 = ONE
      IF ( K .LT. J ) THEN
         S(J-1) = -(ALF(J) - THET)*S(J)/BET(J)
         SUM2 = SUM2 + S(J-1)**2
      END IF
      DO 10 I = J-1,K+1,-1
         S(I-1) = -((ALF(I) - THET)*S(I) + BET(I+1)*S(I+1))/BET(I)
C
C....    SCALE TO AVOID OVERFLOW
C
         IF ( ABS(S(I-1)) .GT. BIG ) THEN
            F = ONE/S(I-1)
            S(I-1) = ONE
            CALL DSCAL(J-I+1,F,S(I),1)
            SUM2 = (SUM2*F)*F
         END IF
         SUM2 = SUM2 + S(I-1)**2
10    CONTINUE
C
C.... STOP EXECUTION GRACEFULLY IF S(K) IS EXACTLY ZERO
C
      IF (S(K) .EQ. ZERO) THEN
         IERR = IERR + 256
```

```
      RETURN
   ENDIF
   F = ONE/S(K)
   S(K) = ONE
   SUM2 = (SUM2*F)*F
   CALL DSCAL(J-K,F,S(K+1),1)
   IF ( K .LE. 1 ) GO TO 30

   FORWARD RECURRENCE

      X = ZERO
      S(1) = ONE
      SUM1 = ONE
      IF ( K .GT. 2 ) THEN
         S(2) = -((ALF(1) - THET)*S(1))/BET(2)
         SUM1 = SUM1 + S(2)**2
         DO 20 I = 2,K-2
            S(I+1) = -((ALF(I) - THET)*S(I) + BET(I)*S(I-1))/BET(I+1)
            IF ( ABS(S(I+1)) .GT. BIG ) THEN
               F = ONE/S(I+1)
               S(I+1) = ONE
               CALL DSCAL(I,F,S,1)
               SUM1 = (SUM1*F)*F
            END IF
            SUM1 = SUM1 + S(I+1)**2
         CONTINUE
         X = BET(K-1)*S(K-2)
      END IF
      X = -(X + (ALF(K-1) - THET)*S(K-1))/BET(K)

      MATCH X WITH S(K)

      IF ( X .EQ. ZERO ) THEN
         RES = ONE
         RETURN
      END IF
      F = S(K)/X
      CALL DSCAL(K-1,F,S,1)
      SUM2 = SUM2 + SUM1*F**2
      RES = BET(K)*S(K-1)

   NORMALIZE S

   F = ONE/SQRT(SUM2)
   CALL DSCAL(J,F,S,1)
   RES = RES*F + (ALF(K) - THET)*S(K)
   IF ( K .LT. J ) RES = RES + BET(K+1)*S(K+1)
   COR = S(K)*RES
   RES = ABS(RES)

   RETURN
   END
```

Subroutine SUBTJ

In theory, unreduced tridiagonal matrices do not have equal eigenvalues, but in practice they can have two eigenvalues that are so close to one another that they are indistinguishable in finite precision. This creates certain difficulties when computing eigenvectors with close eigenvalues. The problem is solved by working with a submatrix $T_{l,m}$ of the tridiagonal T_j. This routine obtains an estimate of the indices l and m that defines a submatrix for which EIG(I) is simple. INFO(I) = N×K + J stores the two indices J and K. J is the step at which the eigenvector corresponding to EIG(I) was computed and K is the index of the element of this eigenvector with largest absolute value. When INFO(I) is negative it indicates that the Ritz vector in Y(I) is converged.

```
      SUBROUTINE SUBTJ(ALF,BET,EIG,INFO,TOL,I,N,NEIG,J,L,K,M,
     1                 U,IERR)

   THIS ROUTINE SCANS BACK THROUGH THE CONVERGED EIGENVALUES
   FOR COPIES OF EIG(I) TO DETERMINE THE SUBMATRIX T(M,L) AND
   THE RIGHT HAND SIDE E(K) FOR THE GIVENS RECURRENCE.

   INPUTS
```

```
C
C     EIG(NEIG)     LIST OF THE CONVERGED EIGENVALUES
C     INFO(NEIG)    INFORMATION ABOUT THE EIGENVECTORS
C     TOL           TOLERANCE FOR FINDING COPIES OF EIG(I)
C     I             INDEX OF THE EIGENVALUE CONSIDERED
C     N             DIMENSION OF THE EIGENPROBLEM
C     NEIG          NUMBER OF EIGENVALUES IN EIG
C
C.... OUTPUTS
C
C     K             INDEX OF THE RIGHT HAND SIDE FOR GIVENS
C     L             INDEX OF THE LAST ELEMENT OF THE SUBMATRIX
C     M             INDEX OF THE FIRST ELEMENT OF THE SUBMATRIX
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION ALF(1),BET(1),EIG(1),INFO(1),U(1)
C
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
      L = J
      EIGI = EIG(I)
C
C.... CHECK SPECIAL CASE OF EIGI = ALL ALTERNATE ALF'S
C
      DO 10 JM = J,1,-2
         IF (ABS(ALF(JM)-EIGI) .GE. EPS*SPREAD) GO TO 20
10    CONTINUE
C
C.... FALSE RITZ VALUE ACCEPTED, STOP EXECUTION GRACEFULLY.
C
      IERR = IERR + 2048
      RETURN
C
C.... RUN RECURRENCE UNTIL "PIVOT = DIAGONAL" TO FIND M
C
20    U(JM) = (ALF(JM)-EIGI)*(ONE - TEN*EPS)
      UMIN  = ABS(U(JM))
      IUMIN = JM
      DO 30 M = JM-1,1,-1
         U(M) = ALF(M)-EIGI-BET(M+1)**2/U(M+1)
         IF (ABS(U(M)).LT.TOL*EIGHTH) GO TO 40
         IF (ABS(U(M)).LE.UMIN) THEN
            UMIN = ABS(U(M))
            IUMIN = M
         ENDIF
30    CONTINUE
C
C.... TEMPORARILY
C
      M = IUMIN
C
C.... NOW SET K
C
40    IF (M .EQ. 1) THEN
         K = M
      ELSE
         K = M+1
      ENDIF
C
C.... NOW CHECK FOR CLOSE EIGENVALUES
C
      IF (M .GT. 1) THEN
         DO 50 I1 = I-1,1,-1
            IF (ABS(EIGI-EIG(I1)).LT.ONE28*EPS*SPREAD)
     1          GO TO 60
50       CONTINUE
C
         RETURN
C
60       KPREV = MOD(ABS(INFO(I1)),N)
         IF (KPREV .LE. K) K = KPREV+1
      ENDIF
C
C.... FIND L
C
      DO 200 I2 = I+1,NEIG
         IF (ABS(EIGI - EIG(I2)) .LT. TOL) GO TO 210
200   CONTINUE
210   IF (I2 .LE. NEIG) THEN
         L = ABS(INFO(I2))/N - 1
         IF (INFO(I2) .EQ. 0) L = MOD(ABS(INFO(I)),N)
      END IF
C
      RETURN
      END
```

Subroutine ANALZT

The goal of this routine is to obtain the smallest possible interval that contains a single eigenvalue of T_j (see [24] for more details). It makes full use of the corresponding information that was obtained at the previous step. When J = 2 it simply computes the eigenvalues of the 2 × 2 tridiagonal matrix. Phase I of the routine updates the data structure (THETA, BJ) containing those eigenvalues of T_{j-1} that are about to converge to eigenvalues of the big problem. Their residual bounds go in BJ. Phase II of the routine appends some new items to this data structure. Any converged eigenvalue is removed from THETA, put into EIG, and then explicitly deflated from the tridiagonal. This routine is called by LANSEL. For a more detailed description see [24].

```
      SUBROUTINE ANALZT(J,ALF,BET2,EIG,TAU,OLDTAU,RHO,INFO)
C     THIS ROUTINE UPDATES SOME EIGENVALUES OF A TRIDIAGONAL T(J) USING
      THE EIGENVALUES OF T(J-1).
C     INPUTS
C
C     J           ORDER OF THE TRIDIAGONAL T.
C     ALF         DIAGONAL OF T.
C     BET2        SQUARES OF THE OFFDIAGONAL TERMS, BET2(1) = ZERO
C     EIG         ARRAY OF CONVERGED EIGENVALUES
C     TAU         ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J
C     OLDTAU      ORTHOGONALITY ESTIMATE OF RITZ VECTORS AT STEP J-1
C     RHO         WORKING ARRAY USED IN DEFLAT
C     INFO        INFORMATION ARRAY ABOUT EIGENVECTORS OF T
C
C     INTERNAL VARIABLES
C
C     THET        EXTERIOR EIGENVALUES OF T, NEARLY CONVERGED
C                 RITZ VALUES. THET(1)=LEFTMOST, THET(NDST)=RIGHTMOST.
C     NDST        SIZE OF THET AND BJ
C     BJ          ERROR BOUND ON THET
C                 BJ(I) IS SET TO -1 IF THET(I) DISAPPEARS.
C     NBD         CONTAINS L AND R IN THE TEXT.
C     SPREAD      THET(NDST) - THET(1)
C     EPS         PRECISION OF ARITHMETIC OPERATIONS
C     IP          IP =1 FOR UPDATING LEFT END, IP = 2 FOR THE RIGHT END.
C     INC         INC=1 FOR UPDATING LEFT END, INC=-1 FOR THE RIGHT END.
C     IS          STARTING INDEX (EITHER 1 OR NDST)
C     START       LEFT BOUND ON EIGENVALUES (INC=1), RIGHT BOUND (INC=-1)
C     PROBE       THE OUTER END OF THE NEXT SUBINTERVAL TO BE UPDATED.
C     INDXOK      TRUE, IF THERE ARE I-INC RITZ VALUES EXTERIOR TO THE
C                 NEW THET(I).
C
C     SUBROUTINES: DEFLAT,MOVE1,NEWCOR,NUMLES
C
      IMPLICIT DOUBLE PRECISION(A-H,O-Z)
      PARAMETER (NMAX = 128)
C
      DIMENSION ALF(1),BET2(1),EIG(1),TAU(1),OLDTAU(1),RHO(1),INFO(1)
      LOGICAL INSERT,INDXOK,APPEND
C
      COMMON /RDATA/ RNM,RNM2,SPREAD,TOL,EPS,EPS1,REPS
      COMMON /ATDATA/ THET(NMAX),BJ(NMAX),WINDOW,NBD(2),NDST
      COMMON /IDATA/ EIGL,EIGR,NEIG
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
      IF (J.LE.1) RETURN
      IF (J.EQ.2) THEN
         NDST = 16
         WINDOW = FOURTH/SQRT(REPS)
         THET(1) = (ALF(1) + ALF(2) - SQRT(FOUR*BET2(2) +
     1              (ALF(1) - ALF(2))**2))*HALF
         THET(NDST) = ALF(1) + ALF(2) - THET(1)
         BJ(1) = SQRT(RNM2/(ONE + BET2(2)/(THET(1) - ALF(1))**2))
         BJ(NDST) = SQRT(RNM2/(ONE + BET2(2)/(THET(NDST) - ALF(1))**2))
         NBD(1) = 1
         NBD(2) = NDST
         SPREAD = THET(NDST) - THET(1)
         RETURN
      END IF
      SPREAD = THET(NDST) - THET(1)
```

```
      TOL = TWO*REPS*SPREAD
      W = WINDOW*TOL
C
C.... BEGIN PHASE 1.
C
C.... LOOP FOR LEFT END, THEN RIGHT
C
      DO 100 IP = 1,2
         INC = 3 - 2*IP
         IS = (NDST-1)*IP - (NDST-2)
         I = IS
         INSERT = .FALSE.
         START = (THET(I) + ALF(J) - INC*SQRT(BET2(J)*FOUR +
     1           (ALF(J) - THET(I))**2))*HALF
         PROBE = THET(I) - INC*BJ(I)
         INDXOK = NUMLES(ALF,BET2,PROBE,J,INC,EPS) .EQ. 0
C
         DO 50 IDUMMY =1,NDST
C
            IF (I-NBD(IP).EQ.INC) GO TO 100
C
C....       EXAMINE I-TH SUBINTERVAL
C
            IF (INDXOK) THEN
               IF (INSERT) THEN
                  START = THET(I)
                  THET(I) = START + INC*MIN(B**2/
     1                 ABS(START-THET(I-INC)),B)
               ELSE
                  IF (INT(SIGN(ONE,PROBE-START)).EQ.INC) START = PROBE
               END IF
C
C....          CHECK FOR DISJOINT SUBINTERVALS
C
               IF (I .EQ. NBD(IP)) THEN
                  PROBE=THET(I)+INC*(THET(NBD(2))-THET(NBD(1)))/(FOUR*J)
               ELSE
                  PROBE = THET(I+INC) - INC*BJ(I+INC)
               END IF
               IF (INT(SIGN(ONE,PROBE-THET(I))).EQ.INC) THEN
C
C....             CHECK FOR AN EXTRA RITZ VALUE
C
                  K = NUMLES(ALF,BET2,PROBE,J,INC,EPS)
C
                  IF (K.LT. ABS(I-IS+INC)) THEN
C
C....                THET(I) DISAPPEARS
C
                     BJ(I) = -ONE
                  ELSE
C
C....                RECORD INDXOK FOR NEXT LOOP. USE REFINED BOUNDS.
C
                     IF (.NOT.INSERT) THEN
                        B = BJ(I)
                        INDXOK = (K .LE. ABS(I-IS+INC))
                        BND = MIN(B**2/ABS(PROBE-THET(I)),B)
                        IF (INDXOK.AND.BND.LT.ABS(THET(I)-START)) THEN
                           START = THET(I) - INC*BND
                        END IF
                     END IF
                  END IF
               END IF
            ELSE
C
C....          PREPARE FOR AN INTRUDING RITZ VALUE
C
               IF ((IS.EQ.NBD(IP).OR.BJ(NBD(IP)-INC).LT.W).AND.
     1              NBD(2)-NBD(1).GT.1) NBD(IP) = NBD(IP) + INC
               CALL MOVE1(THET,I,NBD(IP),-INC,PROBE)
               CALL MOVE1(BJ,I,NBD(IP),-INC,TWO*TOL)
               INSERT = .TRUE.
               INDXOK = .TRUE.
            END IF
C
            IF (BJ(I).GT.TOL) THEN
C
C....          USE NEWTON ITERATION TO FIND NEW THET(I)
C
               CALL NEWCOR(ALF,BET2,START,THET,BJ,INC,I,J,NDST)
C
            END IF
C
            IF (BJ(I).LT.0) THEN
C
```

```
            THET(I) DISAPPEARS

            CALL MOVE1(THET,NBD(IP),I,INC,ZERO)
            CALL MOVE1(BJ,NBD(IP),I,INC,ZERO)
            NBD(IP) = NBD(IP) - INC
            INSERT = .FALSE.
            INDXOK = .TRUE.


            I = I - INC
         END IF
         I = I + INC
      CONTINUE
.     END OF PHASE 1
   CONTINUE
.  BEGIN PHASE 2.
.  APPEND MORE RITZ VALUES AND CHECK FOR CONVERGED RITZ VALUES.
   DO 200 IP = 1,2
      INC = 3 - 2*IP
      IS  = (NDST-1)*IP - (NDST-2)
      I = IS
      DO 150 IDUMMY = 1,J
         NREM = J - NBD(1) - ((NDST+1) - NBD(2))
         IF ((I-NBD(IP))*INC.GT.0) GO TO 200
         APPEND = I.EQ.NBD(IP).AND.(BJ(I).LT.W.OR.(J.EQ.4
     1               .AND.NBD(IP).EQ.IS)).AND.NREM.GT.0
         IF (APPEND) THEN
            START = THET(I) + FLOAT(INC)*BJ(I)
            PROBE = INC*(THET(NBD(2)) - THET(NBD(1)))/NREM
         END IF
         IF (BJ(I) .LE. TOL) THEN

            APPLY QR ALGOR. TO DEFLATE

            CALL DEFLAT(ALF,BET2,THET(I),J)

            INSERT THET(I) INTO EIG

            NEIG = NEIG + 1
            IF (IP.EQ.1) THEN
               EIGL = MAX(EIGL,THET(I))
            ELSE
               EIGR = MIN(EIGR,THET(I))
            END IF
            EIG(NEIG) = THET(I)
            INFO(NEIG) = 0

            REMOVE STABILIZED RITZ VALUES

            CALL MOVE1(THET,NBD(IP),I,INC,ZERO)
            CALL MOVE1(BJ,NBD(IP),I,INC,ZERO)
            NBD(IP) = NBD(IP) - INC
            I = I - INC
         END IF
         IF (APPEND.AND.NBD(2)-NBD(1).GT.1) THEN
            T = START + PROBE
            NBD(IP) = NBD(IP) + INC
            IK = ABS(IS - NBD(IP))
            DO 110 IDUM = 1,J
               IF (NUMLES(ALF,BET2,T,J,INC,EPS).NE.IK) GO TO 120
               T = T + PROBE
            CONTINUE
            THET(NBD(IP)) = T
            START = T - PROBE
            CALL NEWCOR(ALF,BET2,START,THET,BJ,INC,NBD(IP),J,NDST)

         END IF
         IF (J.GT.NDST.AND.I.EQ.NBD(IP).AND.I.NE.IS.AND.BJ(I).GT.
             BJ(I-INC).AND.BJ(I-INC).GT.W) NBD(IP) = NBD(IP) - INC
         I = I + INC
      CONTINUE
   CONTINUE

   RE-ESTABLISH AN END MARKER, IF NECESSARY, AT EARLY STAGE

   DO 300 IP = 1,2
      INC = 3 - 2*IP
      IS  = (NDST-1)*IP - (NDST-2)
      IF (NBD(IP).EQ.IS-INC) THEN
         THET(IS) = THET(NBD(3-IP))
         BJ(IS) = BJ(NBD(3-IP))
```

```
            NBD(IP) = IS
            NBD(3-IP) = NBD(3-IP) + INC
         END IF
300   CONTINUE
C
      RETURN
      END
```

Subroutine NEWCOR

The object of this routine is to find an eigenvalue of T_j in a given interval. It uses a combination of bisection and Newton's method. For a detailed description see [24]. INDEX points to the position in THETA that will eventually hold the computed eigenvalue. On entry THETA(INDEX) contains the eigenvalue of T_{j-1} that is also one end of the interval; ZETA is the other. ZETA is also the starting value for Newton's method. If this interval is too large then the algorithm performs a few steps of bisection to obtain a better estimate, ZETA, to the desired eigenvalue. The eigenvalue counts are obtained by calling NUMLES. Then Newton's iteration is used to compute the eigenvalue to the full computer precision. The Newton correction is computed using a recurrence that is given in [24]. The convergence of Newton's method is guaranteed by deflating the eigenvalues exterior to ZETA implicitly. NEWCOR is called only by ANALZT.

```
C
      SUBROUTINE NEWCOR(ALF,BET2,ZETA,THET,BJ,INC,INDX,J,NDST)
C
C.... COMPUTES EXTERIOR EIGENVALUES OF A TRIDIAGONAL USING A
C     COMBINATION OF BISECTIONS AND NEWTON'S METHOD
C
C.... INPUT
C
C     ALF       DIAGONAL OF T
C     BET2      SQUARES OF THE OFFDIAGONAL TERMS, BET2(1) = ZERO
C     SPREAD    THET(NDST) - THET(1)
C     INC       INC=1 FOR UPDATING LEFT END, INC=-1 FOR THE RIGHTEND.
C     INDX      INDEX OF TO-BE-UPDATED THET.
C     J         ORDER OF THE TRIDIAGONAL T.
C     NDST      SIZE OF THET AND BJ
C
C.... INPUT/OUTPUT
C
C     ZETA      EXTERIOR BOUND FOR EIGENVALUE OF T IN THET[INDX]
C     THET      EXTERIOR EIGENVALUES OF T, NEARLY CONVERGED RITZ VALUES;
C               THET(1)=LEFTMOST, THET(NDST)=RIGHTMOST.
C     BJ        ERROR BOUND ON THET
C
C     SUBROUTINES: NUMLES,QLBOT
C
      IMPLICIT DOUBLE PRECISION(A-H,O-Z)
      PARAMETER (MAXBIS = 15, MAXNEW = 40)
      DIMENSION ALF(1),BET2(1),THET(1),BJ(1)
C
      COMMON /RDATA/ RNM,RNM2,SPREAD,TOL,EPS,EPS1,REPS
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TW056, FIVE12, ORTFAC, OVRFLW
C
      ZOLD = ZETA
      IF (J .EQ. 1) THEN
         ZETA = THET(INDX)
         THET(INDX) = ALF(J)
         BJ(INDX) = SQRT(RNM2)
         RETURN
      END IF
C
C.... PERFORM BISECTION FOR AN IMPROVED ZETA
C
      IS = ((NDST+1) - (NDST-1)*INC)/2
      FACT = FIVE12*FLOAT(J)*LOG(FLOAT(J))
      WIDTH = (THET(INDX) - ZETA)*HALF
      IF (WIDTH .EQ. ZERO) THEN
         WIDTH = BJ(INDX)*HALF
      ENDIF
      IOLD = ABS(IS - INDX)
```

```
      DO 10 IDUMMY = 1,MAXBIS
         IF (ABS(WIDTH)*FACT .LE. ABS(THET((NDST+1)-IS)-THET(INDX)))
     1      GO TO 20
         ZNEW = ZETA + WIDTH
         INEW = NUMLES(ALF,BET2,ZNEW,J,INC,EPS)
         WIDTH = WIDTH*HALF
         IF (INEW .EQ. IOLD) THEN
            ZETA = ZNEW
         ENDIF

      CONTINUE
      CONTINUE

      DO 50 IDUMMY = 1,MAXNEW
         U = ALF(1) - ZETA
         IF ( U .EQ. ZERO ) U = TENTH*EPS*BET2(2)
         RAT = ONE/U
         SUM = RAT
         DO 30 I = 2,J
            H   = BET2(I)/U
            U = ALF(I) - ZETA - H
            IF ( U .EQ. ZERO ) U = TENTH*EPS*(H + BET2(I))
            RAT = ( ONE + H*RAT )/U
            SUM = SUM + RAT
         CONTINUE
         BOT2 = U*SUM

         DEFLATION

         DO 40 I = IS,INDX-INC,INC
            DEL = ZETA - THET(I)
            IF (ABS(DEL).LT.EPS*ABS(ZETA)) THEN
               DEL = EPS*ABS(ZETA)
            ENDIF
            SUM = SUM + ONE/DEL
         CONTINUE

         CHECK FOR CONVERGENCE

         ZNEW = ZETA + ONE/SUM
         ZETA = ZNEW
         IF (SPREAD+TENTH/SUM.EQ.SPREAD.OR.
     1      FLOAT(INC)/SUM.LT.ZERO) THEN
            GO TO 60
         ENDIF
      CONTINUE
      CONTINUE

      CALL QLBOT(ALF,BET2,ZETA,BOT2,J)

      ZNEW = THET(INDX)
      THET(INDX) = ZETA
      ZETA = ZNEW
      BJ(INDX) = SQRT(RNM2*BOT2)

      RETURN
      END
```

Subroutine DEFLAT

The subroutine DEFLAT deflates the tridiagonal matrix T_j using an eigenvalue THETA. One step of the QR algorithm is used to perform the deflation. Each of the last few QR rotations will be almost a row and column interchange. For a more detailed description of the algorithm see [24]. ANALZT is the only routine that calls DEFLAT.

```
      SUBROUTINE DEFLAT(ALF,BET2,THET,J)
C.. THIS ROUTINE PERFORMS DEFLATION OF T USING SHIFT THET.
C.. INPUTS
C     ALF(J)    DIAGONALS OF T
C     BET2(J)   SQUARES OF OFF-DIAGONALS OF T
C     THET      EIGENVALUE OF T TO BE DEFLATED EXPLICITLY
C     J         DIMENSION OF THE TRIDIAGONAL MATRIX
C.. OUTPUTS
C     ALF(J-1) MODIFIED DIAGONALS OF T
```

```
C     BET2(J-1) MODIFIED OFF-DIAGONALS OF T
C
      IMPLICIT DOUBLE PRECISION(A-H,O-Z)
      PARAMETER (MAXITR = 3)
      DIMENSION ALF(J),BET2(J)
C
      COMMON /RDATA/ RNM,RNM2,SPREAD,TOL,EPS,EPS1,REPS
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW
C
C.... PWK ALGORITHM
C
      DO 200 LOOP=1,MAXITR
         C = ONE
         S = ZERO
         G = ALF(1) - THET
         P = G**2
         DO 100 I=1,J-1
            B = BET2(I+1)
            R = P + B
            BET2(I) = S*R
            OLDC = C
            C = P/R
            S = B/R
            OLDG = G
            A = ALF(I+1)
            G = C*(A - THET) - S*OLDG
            ALF(I) = OLDG + (A - G)
            IF ( C .EQ. ZERO ) THEN
               P = OLDC*B
            ELSE
               P = G*(G/C)
            END IF
100      CONTINUE
         BET2(J) = S*P
         ALF(J) =  G + THET
         IF (BET2(J) .LE. EPS*SPREAD) THEN
            GOTO 300
         ENDIF
200   CONTINUE
300   J = J - 1
C
      RETURN
      END
```

Function NUMLES

This routine performs the LDL^T factorization of the tridiagonal matrix T_j to obtain eigenvalue counts for $T_j - \zeta$. By Sylvester's inertia theorem [21], D has the same signature as $T_j - \zeta$. L is not needed and therefore is not computed. The diagonal elements of D are not preserved, but a record of the number of negative or positive terms is kept and returned in NUMLES. NUMLES is called by ANALZT and NEWCOR.

```
C
      INTEGER FUNCTION NUMLES(ALF,BET2,ZETA,N,INC,EPS)
C
C.... ROUTINE TO PERFOM THE SPECTRUM SLICING OF A TRIDIAGONAL MATRIX.
C
C     IF INC =  1, NUMLES RETURNS THE NUMBER OF EIGENVALUES BELOW ZETA.
C     IF INC = -1, NUMLES RETURNS THE NUMBER OF EIGENVALUES ABOVE ZETA.
C
C.... INPUTS
C
C     ALF(N)    DIAGONALS OF T
C     BET2(N)   SQUARE OF THE OFF-DIAGONALS OF T
C     ZETA      THE SHIFT TO BE APPLIED TO T
C     N         DIMENSION OF THE TRIDIAGONAL MATRIX
C     INC       INDEX TO INDICATE ABOVE OR BELOW
C     EPS       COMPUTER PRECISION
C
C.... OUTPUTS
C
C     NUMLES    THE NUMBER OF EIGENVALUES ABOVE/BELOW ZETA.
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      DIMENSION ALF(1),BET2(1)
C
```

```
      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW

      SAVE = BET2(1)
      BET2(1) = ZERO
      DEL = ONE
      K = 0
      DO 10 J=1,N
         DEL = (ALF(J) - ZETA) - BET2(J)/DEL
         IF ( DEL .EQ. ZERO ) DEL = EPS*BET2(J+1)*INC
         IF ( DEL .LT. ZERO ) K = K + 1
   10 CONTINUE

      NUMLES = K
      IF ( INC .LT. 0 ) NUMLES = N - K
      BET2(1) = SAVE

      RETURN
      END
```

Subroutine QLBOT

This routine computes the bottom element of the eigenvector of the tridiagonal matrix T_j corresponding to the eigenvalue contained in THET. The algorithm performs a step of the QL factorization. The product of the sines of the rotation angles is the desired bottom element and is returned in BOT. NEWCOR is the only routine that calls QLBOT.

```
      SUBROUTINE QLBOT(ALF,BET2,THET,BOT,J)

      COMPUTE THE BOTTOM ELEMENT OF THE NORMALIZED EIGENVECTOR OF A
      TRIDIAGONAL MATRIX CORRESPONDING TO EIGENVALUE THET.

      INPUTS

      ALF(J)    DIAGONALS OF T
      BET2(J)   SQUARE OF THE OFF-DIAGONALS OF T
      THET      EIGENVALUE OF T
      J         DIMENSION OF THE TRIDIAGONAL MATRIX

      OUTPUTS

      BOT       BOTTOM ELEMENT OF THE NORMALIZED EIGENVECTOR

      IMPLICIT DOUBLE PRECISION(A-H,O-Z)
      DIMENSION ALF(1),BET2(1)

      COMMON /LANCON/ ZERO, TENTH, EIGHTH, FOURTH, HALF, ONE, TWO,
     1                FOUR, TEN, ONE28, TWO56, FIVE12, ORTFAC, OVRFLW

      BOT = ONE
      C = ONE
      S = ZERO
      G = ALF(J) - THET
      P = G**2

      DO 100 I=J-1,1,-1
         B = BET2(I+1)
         R = P + B
         OLDC = C
         C = P/R
         S = B/R
         OLDG = G
         A = ALF(I)
         G = C*(A - THET) - S*OLDG
         IF ( C .EQ. ZERO ) THEN
            P = OLDC*B
         ELSE
            P = G**2/C
         END IF
         BOT = BOT*S
  100 CONTINUE

      RETURN
      END
```

Subroutine MOVE1

This routine inserts the value of T into the Kth location of the array Y. Elements o Y are shifted up or down depending on the sign of MINC. MOVE1 is only called b ANALZT for data management.

```
C
      SUBROUTINE MOVE1(Y,K,L,MINC,T)
C
C.... MOVES THE CONTENT OF Y TO OPEN A SPACE FOR T.
C
C.... INPUT/OUTPUT
C
C     Y      THE ARRAY TO TO BE REORGANIZED
C     K      THE POSITION IN Y OF THE NEW ELEMENT T
C     L      END OF THE DATA IN Y
C     MINC   THE INCREMENT +1 OR -1
C     T      THE NEW ELEMENT TO BE INSERTED
C
      IMPLICIT DOUBLE PRECISION(A-H,O-Z)
      DIMENSION Y(1)
C
      DO 100 I=L,K-MINC,MINC
         Y(I) = Y(I+MINC)
100   CONTINUE
      Y(K) = T
C
      RETURN
      END
```

REFERENCES

Section 10.1

1. K. J. Bathe, *Finite Element Procedures in Engineering Analysis*. Englewood Cliffs, N. J. Prentice-Hall, 1982.
2. B. Noble, *Applied Linear Algebra*. Englewood Cliffs, N.J.: Prentice-Hall, 1969.

Section 10.2

3. E. L. Wilson, "The Static Condensation Algorithm," *International Journal for Numerica Methods in Engineering*, 8 (1974), 199–203.

Section 10.4

4. B. M. Irons, "Eigenvalue Economisers in Vibration Problems," *Journal of the Roya Aeronautical Society*, 67 (1963), 526.
5. B. M. Irons, "Structural Eigenvalue Problems: Elimination of Unwanted Variables," *AIAA Journal*, 3 (1965), 961.
6. R. J. Guyan, "Reduction of Stiffness and Mass Matrices," *AIAA Journal*, 3 (1965), 380.
7. R. D. Henshell and J. H. Ong, "Automatic Masters for Eigenvalue Economization," *Earthquake Engineering and Structural Dynamics*, 3 (1975), 375–383.

Section 10.5

8. K. J. Bathe, *Finite Element Procedures in Engineering Analysis*. Englewood Cliffs, N. J. Prentice-Hall, 1982.
9. E. L. Wilson, "Numerical Methods for Dynamic Analysis," *International Symposium on Numerical Methods in Offshore Engineering*, Swansea, January 11–15, 1977.

Section 10.6

10. J. J. Dongarra, C. B. Moler, J. R. Bunch, and G. W. Stewart, *LINPACK Users' Guide*, SIAM, Philadelphia, 1979.
11. T. Ericsson and A. Ruhe, "The Spectral Transformation Lanczos Method for the Numerical Solution of Large Sparse Generalized Symmetric Eigenvalue Problems," *Mathematics of Computation*, 35 (1980), 1251–1268.
12. G. Golub, R. Underwood, and J. H. Wilkinson, "The Lanczos Algorithm for the Symmetric $Ax = \lambda Bx$ Problem," Tech. Rep. STAN-CS-72-720, Computer Science Department, Stanford University, 1972.
13. J. Grcar, "Analyses of the Lanczos Algorithm and of the Approximation Problem in Richardson's Method," Ph.D. Thesis, University of Illinois at Urbana-Champaign, 1981.
14. C. Lanczos, "An Iteration Method for the Solution of the Eigenvalue Problem of Linear Differential and Integral Operators," *Journal of Research of the National Bureau of Standards*, 45 (1950), 255–281.
15. B. Nour-Omid, B. N. Parlett, and R. L. Taylor, "Lanczos versus Subspace Iteration for Solution of Eigenvalue Problems," *International Journal for Numerical Methods in Engineering*, 19 (1983), 859–871.
16. B. Nour-Omid and B. N. Parlett, "How to Implement the Spectral Transformation," Tech. Rep. PAM-224, Center for Pure and Applied Mathematics, University of California, Berkeley, 1984.
17. B. Nour-Omid and R. W. Clough, "Dynamic Analysis of Structures Using Lanczos Coordinates," *Earthquake Engineering and Structural Dynamics*, 12 (1984), 565–577.
18. C. C. Paige, "Computational Variants of the Lanczos Method for the Eigenproblem," *Journal of the Institute for Mathematics and its Applications*, 10 (1972), 373–381.
19. C. C. Paige, "Error Analysis of the Lanczos Algorithm for Tridiagonalizing a Symmetric Matrix," *Journal of the Institute for Mathematics and its Applications*, 18 (1976), 341–349.
20. C. C. Paige, "Accuracy and Effectiveness of the Lanczos Algorithm for Symmetric Eigenproblems," *Linear Algebra and its Applications*, 34 (1980), 235–258.
21. B. N. Parlett, *The Symmetric Eigenvalue Problem*. Englewood Cliffs, N.J.: Prentice-Hall, 1980.
22. B. N. Parlett and D. Scott, "The Lanczos Algorithm with Selective Orthogonalization," *Mathematics of Computation*, 33, no. 145 (1979), 217–238.
23. B. N. Parlett, H. D. Simon, and L. M. Stringer, "On Estimating the Largest Eigenvalue with the Lanczos Algorithm," *Mathematics of Computation*, 38, no. 157 (1982), 153–165.
24. B. N. Parlett and B. Nour-Omid, "The Use of Refined Error Bounds When Updating Eigenvalues of Tridiagonals," *Linear Algebra and its Applications*, 68 (1985), 179–219.
25. R. L. Taylor, private communication, 1978.
26. D. S. Scott, "Analysis of the Symmetric Lanczos Process," Tech. Rep. ERL-M78/40, Electronics Research Laboratory, University of California, Berkeley, 1978.
27. H. D. Simon, "The Lanczos Algorithm with Partial Reorthogonalization," *Mathematics of Computation*, 42, no. 165 (1984), 115–142.
28. H. D. Wilkinson, *The Algebraic Eigenvalue Problem*. Oxford: Clarendon, 1965.
29. E. L. Wilson, M. Yuan, and J. M. Dickens, "Dynamic Analysis by Direct Superposition of Ritz Vectors," *Earthquake Engineering and Structural Dynamics*, 10 (1982), 813–821.